Microsoft

Develop AI cloud solutions on Azure [AI-200]

1

Let's have a great time together-



We all contribute to a great class

- This placeholder is designed with bulleted text
- It's easy to remove or add bullets
- Be on time (class hours)
- Be present (avoid mail, calls, side meetings)



What you should know about our facilities

- This placeholder is designed with bulleted text
- This section can be removed if the class is online
- Accommodations
- Internet access
- Parking
- Restrooms
- Emergency

2

About this course

Cloud developer role

Azure AI Cloud Developers contribute to all phases of implementing AI solutions on Azure, with an emphasis on back-end services and components. They are responsible for supporting all phases of the development lifecycle, including requirements gathering, design, development, deployment, security, and monitoring.

Prerequisites

- Fundamental proficiency in app containerization.
- Fundamental proficiency in Azure data services.
- Fundamental proficiency in AI agents and models.
- At least two years of experience in Python
- Are familiar with Azure CLI.

3

Course schedule

	Day 1	Day 2	Day 3	Day 4	Day 5
Morning	Learning path 01: Implement container application hosting on Azure	Learning path 03: Deploy and monitor applications on Azure Kubernetes Service	Learning path 05: Develop AI solutions with Azure Database for PostgreSQL	Learning path 07: Integrate backend services for AI solutions	Learning path 09: Observe and troubleshoot apps on Azure
Lunch					
Late afternoon	Learning path 02: Deploy and manage apps on Azure Container Apps	Learning path 04: Develop AI solutions with Azure Cosmos DB for NoSQL	Learning path 06: Enhance AI solutions with Azure Managed Redis	Learning path 08: Manage application secrets and configuration for AI solutions	

4

Get the most out of your Microsoft Learn profile

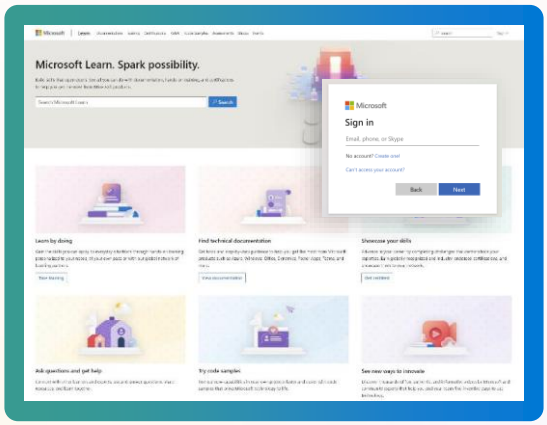
Verify, track, and share your training and certification progress and accomplishments—*all on one platform*.

- Claim your achievement code for this course and share you have completed it.
- Access your course material and track progress on your learning activities.
- Share and verify your Microsoft Credentials via email, on social networking platforms, and on your resume.
- Download and print transcripts and certificates.
- Manage your upcoming activities and credential appointments.

www.aka.ms/MyMicrosoftLearnProfile

Create your Microsoft Learn profile at learn.microsoft.com

- Select *Sign in* at the top, right corner of any Microsoft Learn page.
- Follow the Microsoft account authentication process.
- If the account that you have chosen to sign-in with doesn't already have a Microsoft Learn profile, you'll be guided to create one.



5

Access your course material

All course content is available on Microsoft Learn.

- We'll go through this content together and, as the course progresses, I will advise you on which modules to review.
- You can provide feedback for modules on Microsoft Learn. Find out how at the bottom of each page.

http://learn.microsoft.com/training/courses/browse

This course includes labs:

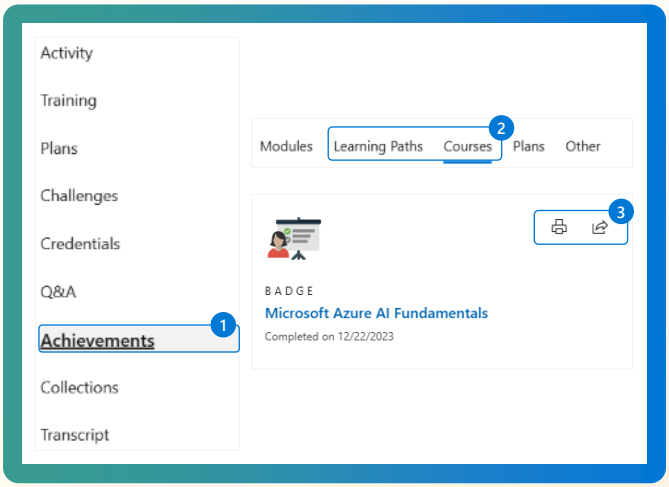
Detailed lab instructions are included in your lab environment.

Need help? See our troubleshooting guide or provide feedback for Microsoft Learn content.

6

Celebrate your accomplishments and feel empowered

Get your achievement recognized and earn your badge for completing this course.



1. Go to **Achievements** on your Microsoft Learn profile.
2. Find your new course competition badge under **Learning Paths and or Courses.**
3. Share your new achievement with your professional network. You can also download and print your certificate.

http://www.aka.ms/MyMicrosoftLearnProfile

7

Become Microsoft Certified

91% of certified IT professionals feel more confident in their abilities after earning certifications[1]

Get recognized by earning industry validation for technical knowledge. Ensure you stay current with the necessary skills and expertise for continued success.

Start at

https://learn.microsoft.com/credentials

Microsoft role-based and specialty certifications require annual renewal.[2]

Learn about certification renewal at

https://aka.ms/RenewYourCert

[1] "2023 Value of IT Certification | Candidate Report." Pearson VUE, 2023 [2] Microsoft fundamentals certifications don't expire

8

Get ready for your Microsoft Credential

Exam AI-200: Developing AI Cloud Solutions on Azure

Understand the skills measured by the credential

Study area	Percentage
Develop containerized solutions on Azure	20–25%
Develop AI solutions by using Azure data management services	25–30%
Connect to and consume Azure services	20–25%
Secure, monitor, and troubleshoot Azure solutions	20–25%

Build confidence in your skills

Find in the credential page resources to help prepare:

- Watch exam prep videos
- Review the exam study guide
- Demo the exam experience with the exam sandbox
- Take a practice assessment

9

Microsoft

Implement container application hosting on Azure

© Copyright Microsoft Corporation. All rights reserved.

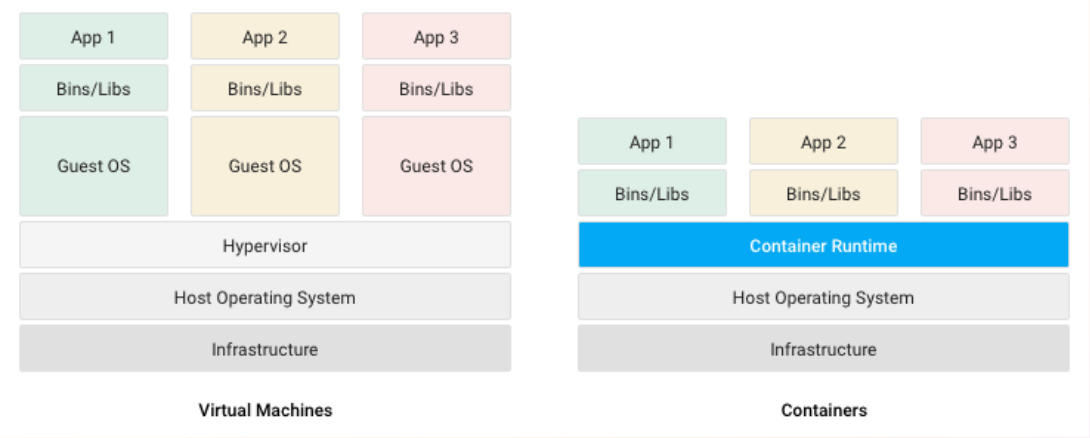
10

Module 1: Store and manage containers in Azure Container Registry

© Copyright Microsoft Corporation. All rights reserved.

11

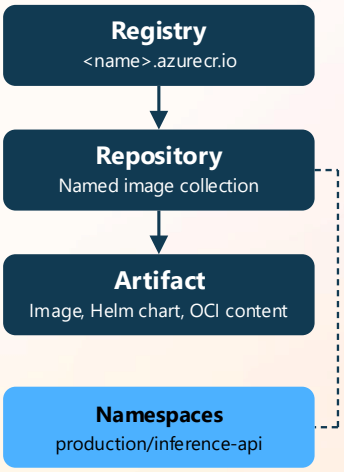
Containers



© Copyright Microsoft Corporation. All rights reserved.

12

Registries, repositories, and artifacts



© Copyright Microsoft Corporation. All rights reserved.

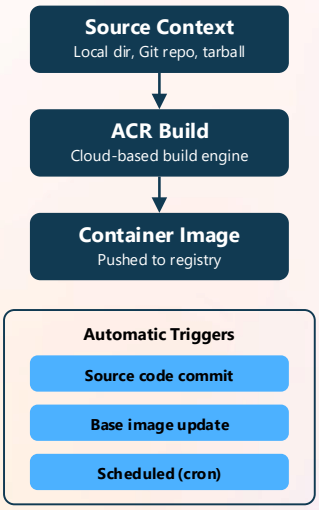
13

Build and run images with ACR Tasks

- Quick tasks build and push in one command
- Source code trigger example

```
az acr task create \
  --registry myregistry \
  --name build-inference-api \
  --image inference-api:{{.Run.ID}} \
  --context https://github.com/myorg/
    inference-api.git#main \
  --file Dockerfile \
  --git-access-token $PAT
```

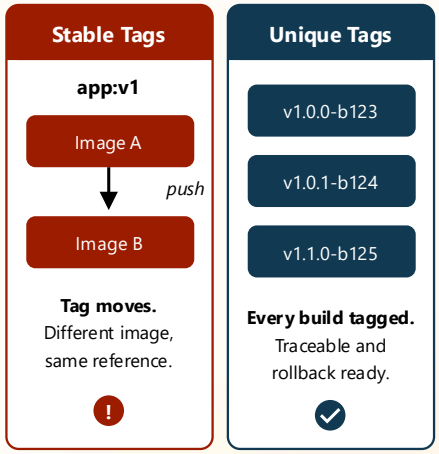
- Multi-step tasks define build-test-push workflows in YAML



© Copyright Microsoft Corporation. All rights reserved.

14

Tag and version images



- Lock production images: Prevents deletion, overwrite, and survives retention policies

```
az acr repository update \
  --name myregistry \
  --image inference-api:v1.2.0 \
  --write-enabled false
```

- Clean up untagged images

```
az acr task create \
  --registry myregistry \
  --name cleanup-untagged \
  --cmd "acr purge --filter '.*:.*' \
  --untagged --ago 7d" \
  --schedule "0 0 * * 0" \
  --context /dev/null
```

- Unique tag patterns: build ID, Git SHA, timestamp, or combined

© Copyright Microsoft Corporation. All rights reserved.

15

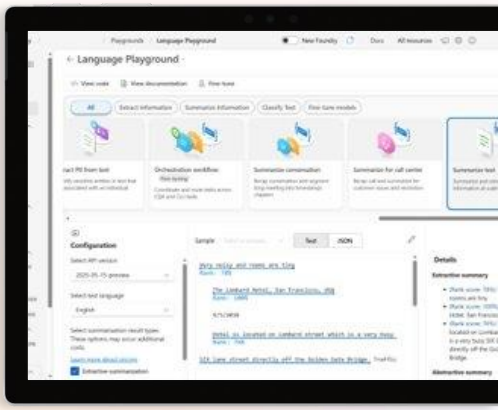
Exercise

Build and run a container image with ACR Tasks

In this exercise, you use Azure Container Registry (ACR) Tasks to build and manage container images entirely in the cloud, without requiring a local Docker installation.

Estimated time: 30 minutes

Start the exercise at:
https://go.microsoft.com/fwlink/?linkid=2346552



© Copyright Microsoft Corporation. All rights reserved.

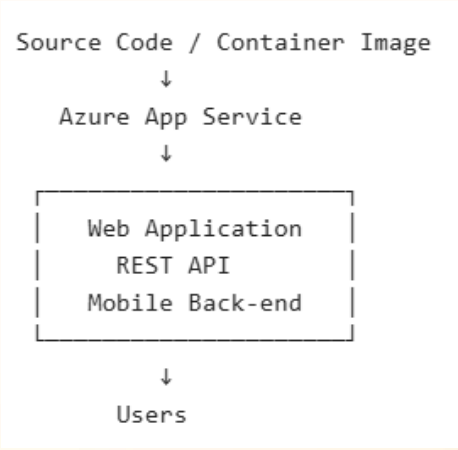
16

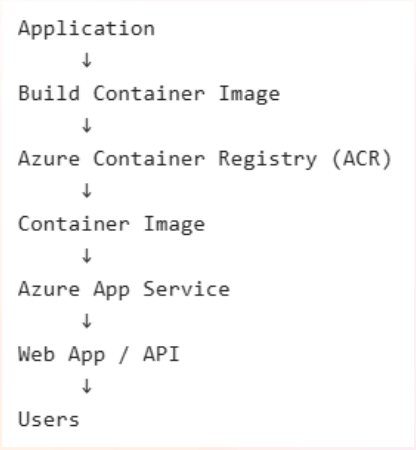
Module 2: Deploy containers to Azure App Service

© Copyright Microsoft Corporation. All rights reserved.

17

Azure App Service Concept





© Copyright Microsoft Corporation. All rights reserved.

18

Deploy containers to Azure App Service

Image sources

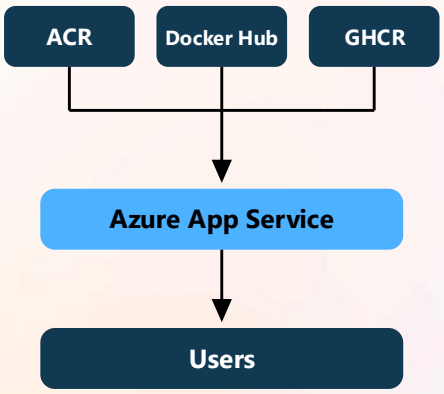
- Azure Container Registry (recommended for production)
- Docker Hub, GitHub Container Registry, self-hosted registries

ACR authentication

- Managed identity (system or user-assigned) with AcrPull role
- Admin credentials for dev/test scenarios

Deployment methods

- Azure portal guided experience
- Azure CLI, VS Code with Docker and App Service extensions
- Continuous deployment via ACR webhook



© Copyright Microsoft Corporation. All rights reserved.

19

Configure container runtime behavior

Startup command

- Overrides Dockerfile CMD (ENTRYPOINT unchanged)
- Use for runtime args, migrations, custom configs

Port configuration

- Set `WEBSITES_PORT` if container listens on a port other than 80 or 8080
- App Service handles TLS termination. Container receives HTTP.

Persistent storage

- Survives restarts, shared across scaled instances

Always-on and health checks

- Always-on prevents ~20 min idle unloading (Basic tier+)
- Health check path pinged every minute. Failed instances removed from rotation.



© Copyright Microsoft Corporation. All rights reserved.

20

Configure application settings

App settings

- Injected as environment variables at container start
- All settings encrypted at rest
- Access via standard env var methods (`os.environ` in Python)

Connection strings

- Type-prefixed env vars (e.g., `SQLAZURECONNSTR_`)
- Use app settings instead for non-.NET apps

Slot settings

- Stick to the slot during swap operations
- Use for environment identifiers, endpoints, feature flags

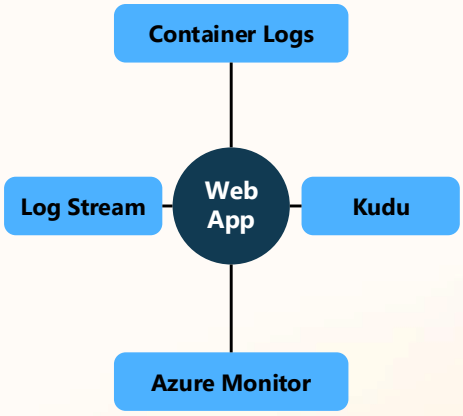
Key Vault references

- `@Microsoft.KeyVault(SecretUri=...)` syntax
- Resolved automatically. App reads plain env vars.
- Requires managed identity with Key Vault access

© Copyright Microsoft Corporation. All rights reserved.

21

Observe and troubleshoot containerized apps



Container logs

- Captures stdout, stderr, platform lifecycle events

Log stream

- Real-time output via CLI (az webapp log tail) or portal
- Shows all instances in scaled-out apps

Kudu (SCM site)

- Environment variable verification
- File system browser for /home/LogFiles/
- Diagnostic dump download

Azure Monitor integration

- Send logs to Log Analytics for KQL queries and alerting
- Categories: ConsoleLogs, HTTPLogs, PlatformLogs

© Copyright Microsoft Corporation. All rights reserved.

22

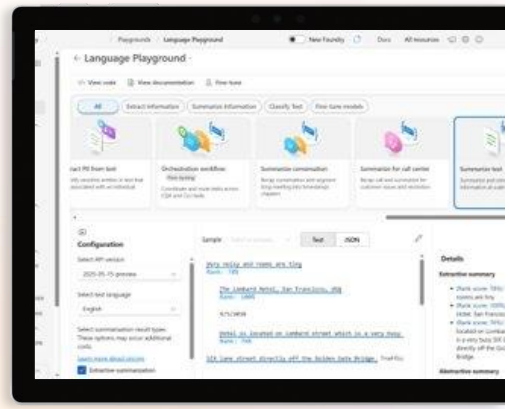
Exercise

Deploy a container to Azure App Service

In this exercise, you deploy a Linux container image from Azure Container Registry (ACR) to Azure App Service.

Estimated time: 30 minutes

Start the exercise at:
https://go.microsoft.com/fwlink/?linkid=2347415



© Copyright Microsoft Corporation. All rights reserved.

23

Microsoft

Deploy and manage apps on Azure Container Apps

© Copyright Microsoft Corporation. All rights reserved.

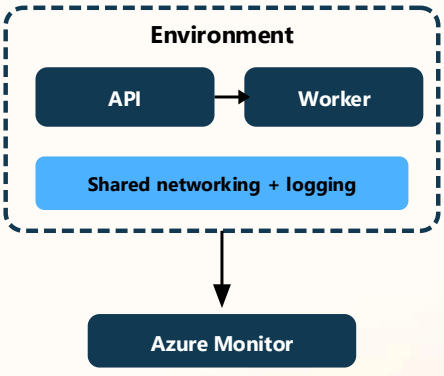
24

Module 1: Deploy containers to Azure Container Apps

© Copyright Microsoft Corporation. All rights reserved.

25

Explore Container Apps environments



What an environment provides

- Shared networking boundary for grouped apps
- Internal DNS for app-to-app communication
- Common logging integration (Azure Monitor)

Design decisions

- **Separate by lifecycle:** Dev, test, prod environments
- **Align to networking:** Group apps that communicate privately
- **Plan observability early:** Choose log destination before production

© Copyright Microsoft Corporation. All rights reserved.

26

Deploy a container app using the Azure CLI and YAML

Three deployment paths

- **`containerapp up`:** Fastest path, creates environment automatically
- **`containerapp create`:** Explicit control, existing environment
- **`--yaml`:** Source-controlled config, reviewable via code review

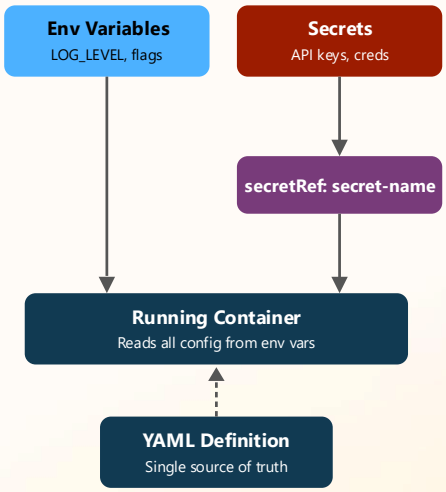
Revisions

- Each update creates a new revision (versioned snapshot)
- Validate new revision before shifting traffic
- Use `containerapp update` for image and config changes

© Copyright Microsoft Corporation. All rights reserved.

27

Configure runtime settings with environment variables and secrets



Environment variables (non-sensitive)

- Log levels, feature flags, dependency URLs
- Same image runs in dev, staging, prod with different values
- Set at create time or add incrementally later

Secrets (sensitive values)

- API keys, credentials, signing keys stored separately
- Rotate secrets without rebuilding or redeploying images
- Key Vault references supported for centralized management

Secret references

- Map secrets to env vars so app code reads them transparently
- Keep secret values out of YAML files and shell history

© Copyright Microsoft Corporation. All rights reserved.

28

Configure image pull authentication for private registries

Username / password

- Works with any registry (ACR, Docker Hub, GHCR)
- Quick to set up for validation and prototyping
- More secrets to manage and rotate over time

Managed identity (recommended for ACR)

- No credentials to store or rotate
- Grant only AcrPull for least privilege
- Best fit for production AI services pulling many images
- Use `--identity system` for ACR

Key consideration: Registry config is separate from app config. Image pull failures often look like auth errors.

© Copyright Microsoft Corporation. All rights reserved.

29

Verify deployments with logs and status

Logs (fastest signal)

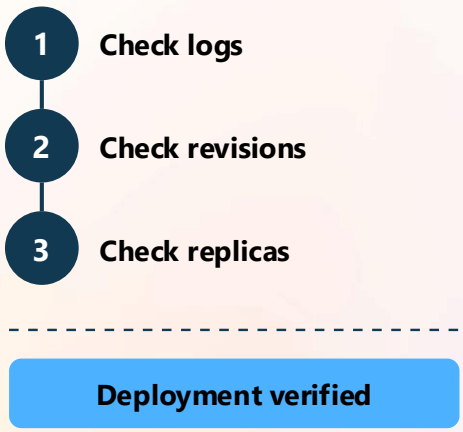
- Console logs surface startup errors, missing env vars, connection failures
- System logs reveal platform-level issues (image pull, scaling)
- Stream logs in real time during active debugging

Revisions (version check)

- Confirm a new revision exists after each update
- Verify the revision reached a healthy, active state
- Include inactive revisions for deeper investigation

Replicas (scaling check)

- Detect scale-to-zero, crash loops, and cold start issues
- Verify expected instance count under load



© Copyright Microsoft Corporation. All rights reserved.

30

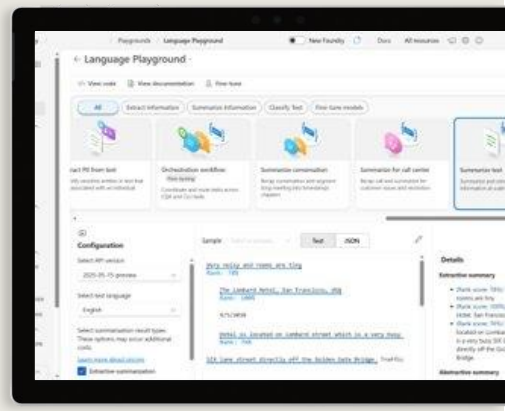
Exercise

Deploy a containerized backend API to Azure Container Apps

In this exercise, you deploy a containerized backend API to Azure Container Apps. You use a managed identity to securely pull images from Azure Container Registry and configure secrets as environment variables.

Estimated time: 30 minutes

Start the exercise at:
https://go.microsoft.com/fwlink/?linkid=2348000



© Copyright Microsoft Corporation. All rights reserved.

31

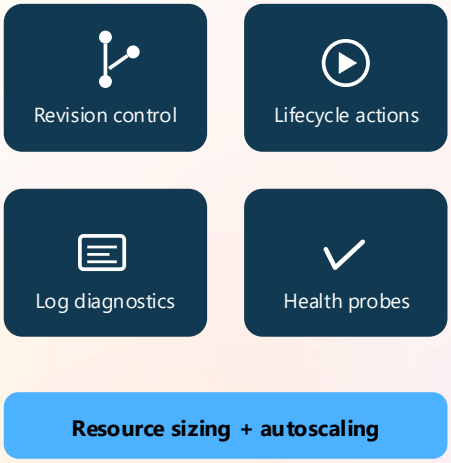
Module 2: Manage containers in Azure Container Apps

© Copyright Microsoft Corporation. All rights reserved.

32

Introduction

- Revision-based model: every update creates a versioned snapshot
- Built-in lifecycle actions for incident response
- Integrated logging with Azure Monitor
- Health probes gate traffic to healthy replicas
- Per-replica sizing and autoscaling for cost control



© Copyright Microsoft Corporation. All rights reserved.

33

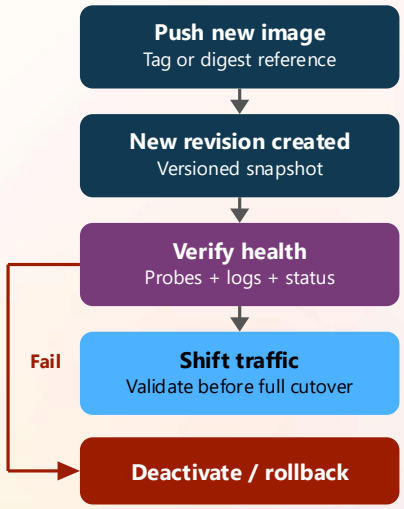
Update images and manage revisions safely

Image identity

- Tags are convenient but mutable. Digests are immutable.
- Use digests in production for traceability during incidents
- Correlate operational behavior with a specific build artifact

Revision workflow

- Each image update creates a new revision
- Verify health before shifting traffic
- Deactivate problematic revisions. Preserve for investigation.
- Define a retention strategy for inactive revisions



© Copyright Microsoft Corporation. All rights reserved.

34

Manage the container app lifecycle

Choose the right action

- **Stop:** Guarantees zero traffic. Broad scope. Use during incidents.
- **Restart:** Clears transient state. Use for deadlocks, stuck processes.
- **Deactivate revision:** Isolates one bad release. Safest first step.

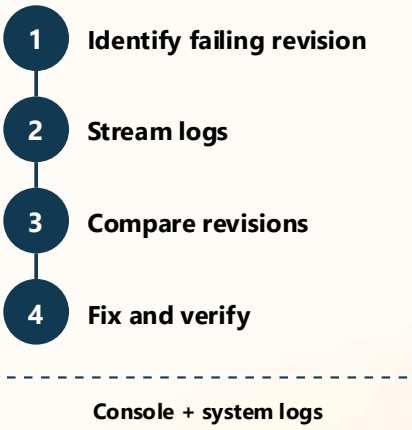
Common failure patterns to validate

- Image pull failures (registry credentials)
- Port mismatches (container vs. ingress config)
- Missing environment variables or secrets
- Probe failures (wrong path, short timeout)
- Resource pressure (CPU throttling, OOM restarts)

© Copyright Microsoft Corporation. All rights reserved.

35

Monitor logs and troubleshoot issues



Log types

- Console logs: Application output (errors, startup messages)
- System logs: Platform events (image pull, scaling, probes)

Repeatable troubleshooting workflow

- Identify which revision is failing
- Stream logs while reproducing the issue
- Compare working vs. failing revision config
- Apply targeted fix, verify next revision is healthy

Structured logging tips

- Include request ID, revision ID, model version
- Log latency breakdowns for dependency calls
- Avoid logging sensitive content (prompts, documents)

© Copyright Microsoft Corporation. All rights reserved.

36

Configure health probes and troubleshoot failures

Readiness vs. liveness

- Readiness: Guards model loading and warmup.
- Liveness: Catches deadlocks. Triggers restart.

Tuning for AI services

- Set `initialDelaySeconds` to cover model load time
- Use generous `failureThreshold` for slow warmup
- Don't let liveness fire during startup
- Probe endpoints should not depend on external services

When probes fail immediately after deploy

- Validate port and path match the container's HTTP server
- Check that timeout allows enough time for response

© Copyright Microsoft Corporation. All rights reserved.

37

Optimize container resources and scaling

Per-replica sizing

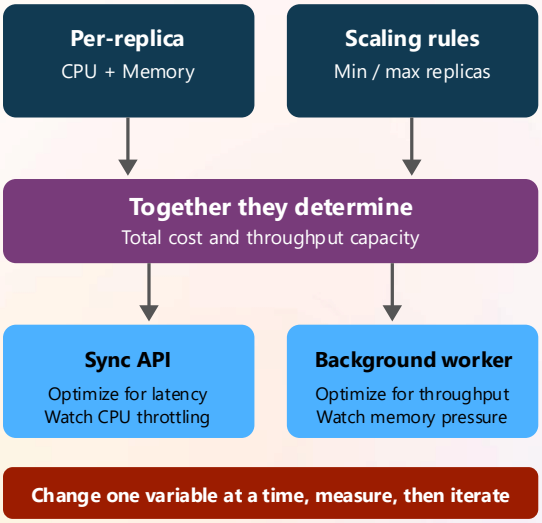
- CPU throttling = need more CPU per replica
- OOM restarts = need more memory per replica
- Total cost = per-replica size x replica count

Scaling strategies differ by workload

- **Sync API:** Minimum replicas to reduce cold starts. Optimize for latency.
- **Background worker:** Scale-to-zero if event-driven. Optimize for throughput.

Optimization approach

- Measure before changing (logs + metrics)
- Change one variable at a time
- Reassess after model version changes



© Copyright Microsoft Corporation. All rights reserved.

38

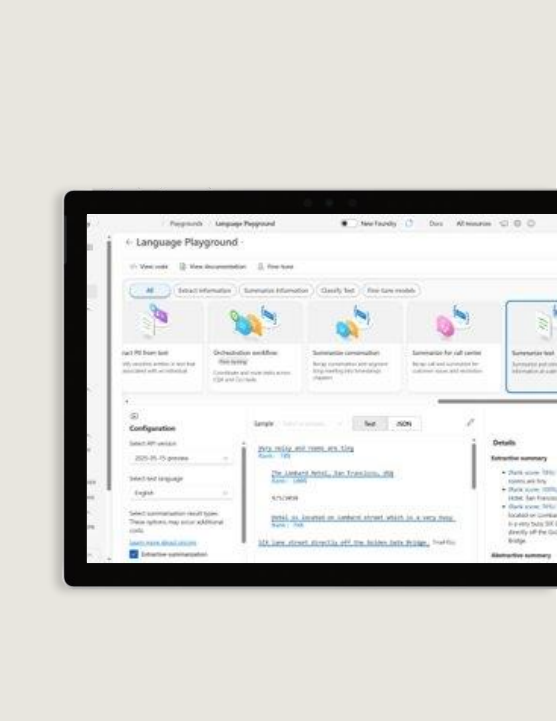
Exercise

Diagnose and fix a failing deployment

In this exercise, you troubleshoot a failing container app and apply targeted fixes. You use revision status, logs, and the Azure CLI to isolate deployment issues.

Estimated time: 30 minutes

Start the exercise at:
https://go.microsoft.com/fwlink/?linkid=2348400



© Copyright Microsoft Corporation. All rights reserved.

39

Module 3: Scale containers in Azure Container Apps

© Copyright Microsoft Corporation. All rights reserved.

40

Configure scale rules

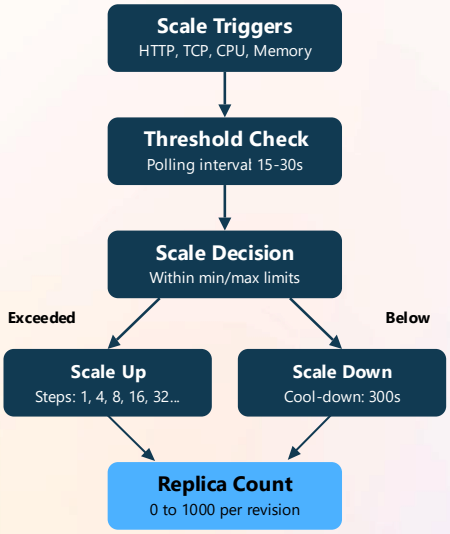
Scale definition components

- Limits: Min and max replica count (0 to 1000)
- Rules: Triggers that drive scaling decisions
- Behavior: Timing for polling, cool-down, stabilization

Built-in rule types

- HTTP: Concurrent requests per replica (default: 10)
- TCP: Concurrent connections for persistent protocols
- CPU: Utilization percentage (cannot scale to zero)
- Memory: Consumption percentage (cannot scale to zero)

Key behavior: Multiple rules use highest replica count



© Copyright Microsoft Corporation. All rights reserved.

41

Implement event-driven scaling with KEDA

Azure-native scalers (first-party)

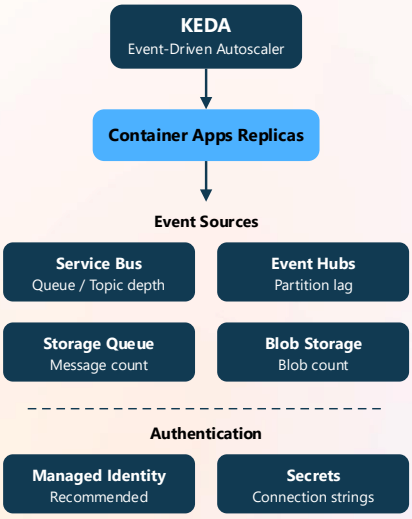
- Service Bus: Scale on queue/topic message count
- Event Hubs: Scale on partition consumer lag
- Storage Queue: Scale on approximate message count
- Blob Storage: Scale on blob count

How it works

- KEDA polls event source every 30 seconds
- Divides current metric by threshold to get replica count
- Supports scale-to-zero for idle queue processors

Authentication

- Managed identity (recommended for production)
- Secrets-based connection strings



© Copyright Microsoft Corporation. All rights reserved.

42

Apply KEDA scalers for custom workloads

Beyond Azure-native scalers

- Kafka: Consumer group lag per partition
- Redis: List length or stream pending entries
- Cron: Time-based pre-scaling for known peaks
- Prometheus: Custom PromQL business metrics

Combining scalers

- Cron establishes baseline capacity on schedule
- HTTP or event scalers handle real-time demand
- Platform uses highest replica count across all rules

Migrating from Kubernetes KEDA

- Map ScaledObject type to Container Apps rule type
- Extract metadata and auth parameters
- Replace TriggerAuthentication with secrets or identity

© Copyright Microsoft Corporation. All rights reserved.

43

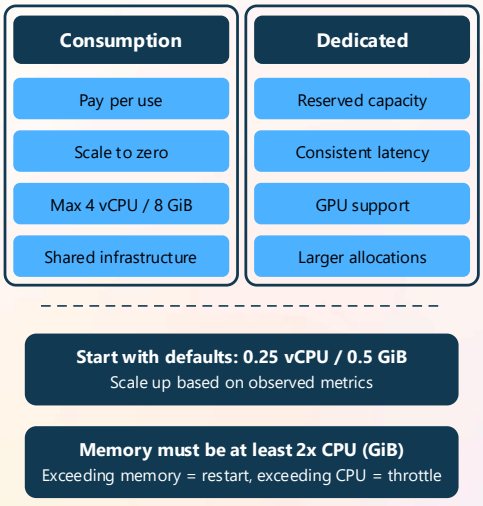
Select compute resources for performance and cost

Resource allocation

- CPU in cores, memory in GiB per container
- Memory must be at least 2x CPU value
- Exceeding memory limit restarts the replica
- Exceeding CPU limit throttles (no restart)

Cost levers

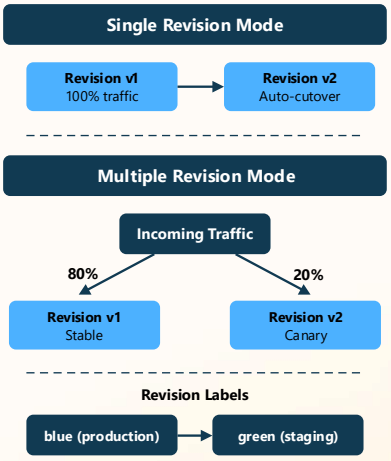
- Scale-to-zero eliminates idle compute charges
- Right-size based on actual utilization, not estimates
- Total capacity = per-replica resources x max replicas
- Smaller replicas scale more precisely but more frequently



© Copyright Microsoft Corporation. All rights reserved.

44

Choose and apply revision modes



Single revision mode (default)

- One active revision at a time
- Zero-downtime deployment is automatic
- New revision replaces old after health checks pass
- No traffic splitting capability

Multiple revision mode

- Multiple revisions active simultaneously
- Traffic splitting by percentage weights
- Canary releases, blue-green deployments, A/B testing
- Must deactivate old revisions manually

Revision labels

- Named endpoints for direct revision access
- Test specific revisions without affecting production traffic
- Enable blue-green switching via label moves

© Copyright Microsoft Corporation. All rights reserved.

45

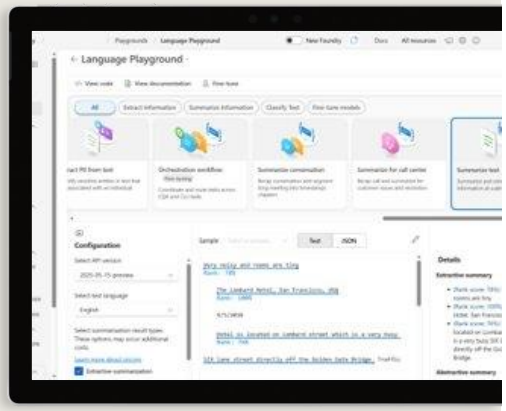
Exercise

Configure autoscaling using KEDA triggers

In this exercise, you deploy a simple mock agent API and configure autoscaling based on HTTP concurrent requests.

Estimated time: 30 minutes

Start the exercise at:
https://go.microsoft.com/fwlink/?linkid=2348401



© Copyright Microsoft Corporation. All rights reserved.

46

Microsoft

Deploy and monitor applications on Azure Kubernetes Service

© Copyright Microsoft Corporation. All rights reserved.

47

Module 1: Deploy applications to Azure Kubernetes Service

© Copyright Microsoft Corporation. All rights reserved.

48

Create Kubernetes deployment manifests

Desired state in YAML

- Define image, replica count, labels, and container port
- Keep Pod labels aligned with selector labels
- Treat the manifest as the source of truth for runtime intent

Runtime sizing

- Requests reserve schedulable CPU and memory
- Limits cap runaway usage and prevent noisy neighbors
- Replica count trades cost for availability and throughput

Configuration flow

- Use env vars for non-sensitive settings
- Reference Secrets for keys and credentials
- Avoid embedding secret values in the manifest

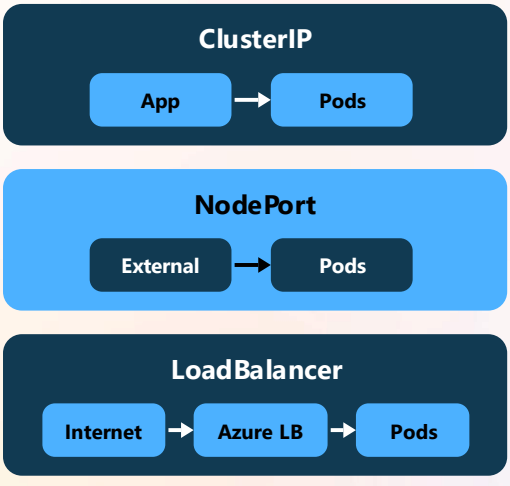
```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ai-api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: ai-api    # must match
  template:
    metadata:
      labels:
        app: ai-api  # must match
    spec:
      containers:
      - name: ai-api
        image: myacr.azurecr.io/ai-api:v1
        # ports, resources, env, secrets
```

© Copyright Microsoft Corporation. All rights reserved.

49

Expose applications in Azure Kubernetes Services

- **ClusterIP:** Internal-only endpoint for app-to-app traffic
  - Best fit for back-end services inside the cluster
- **NodePort:** Opens a port on every node for basic external reachability
  - Useful for development and simple validation
- **LoadBalancer:** Provisions an Azure-managed public entry point
  - Best fit for production APIs serving external clients
- Service selector must match Pod labels
- `port` is the client-facing port
- `targetPort` is the container-facing port



© Copyright Microsoft Corporation. All rights reserved.

50

Deploy applications to Azure Kubernetes Services

Deployment flow

- Apply manifests and let the cluster reconcile state
- Wait for Pods to reach Running and replicas to become Ready
- Confirm the Service has endpoints and an address

Verification order

- Start with Pod and Deployment status
- Check Service exposure and endpoint assignment
- Read logs before changing configuration

Common failure signals

- ImagePullBackOff: Registry path or pull access issue
- CrashLoopBackOff: App starts, then exits or fails health checks
- Pending: Resource requests exceed available node capacity
- No endpoints: Service selector does not match Pod labels

© Copyright Microsoft Corporation. All rights reserved.

51

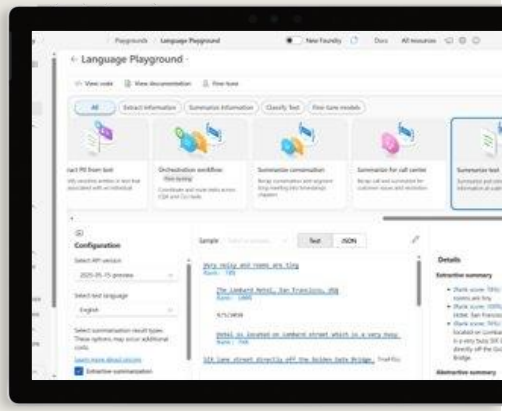
Exercise

Deploy an AI inference API to Azure Kubernetes Service

In this exercise, you deploy Azure resources including a Microsoft Foundry AI model, Azure Container Registry (ACR), and Azure Kubernetes Service (AKS) cluster.

Estimated time: 30 minutes

Start the exercise at:
https://go.microsoft.com/fwlink/?linkid=2345722



© Copyright Microsoft Corporation. All rights reserved.

52

Module 2: Configure applications on Azure Kubernetes Service

© Copyright Microsoft Corporation. All rights reserved.

53

Define ConfigMaps for application settings

Key-value configuration

- Nonsensitive settings stored as string pairs, up to 1 MiB
- Update without rebuilding containers

Consumption patterns

- Environment variables via configMapKeyRef
- Volume mounts for file-based config, auto-update on sync

Immutable option

- Prevents changes, reduces API server load
- Requires delete and recreate to modify

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-settings
data:
  FEATURE_X_ENABLED: "true"
  SERVICE_ENDPOINT:
"https://api.example.com"
  app.config: |
    log_level=info
    timeout_seconds=30
```

© Copyright Microsoft Corporation. All rights reserved.

54

Implement secrets for sensitive data

Kubernetes Secrets

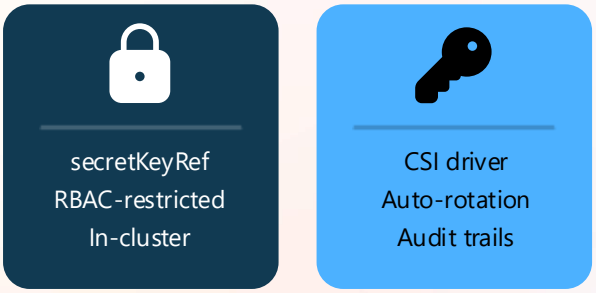
- Store credentials, API keys, and connection strings
- Inject via secretKeyRef or mount as files

Azure Key Vault integration

- CSI driver mounts secrets directly from Key Vault into Pods
- Automated rotation and audit logging

Best practices

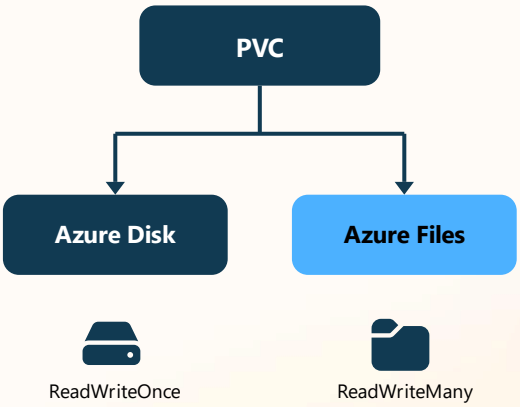
- Restrict access with RBAC, never commit literal values
- Rotate credentials regularly and trigger rollouts



© Copyright Microsoft Corporation. All rights reserved.

55

Attach persistent storage to an app



Storage model

- PVC requests storage with capacity and access mode
- StorageClass dynamically provisions Azure Disk or Azure Files

Access modes

- ReadWriteOnce: single-node block storage (Azure Disk)
- ReadWriteMany: shared file access across Pods (Azure Files)

Verification

- Confirm PVC status is Bound after applying
- Write a test file, delete the Pod, verify data persists

© Copyright Microsoft Corporation. All rights reserved.

56

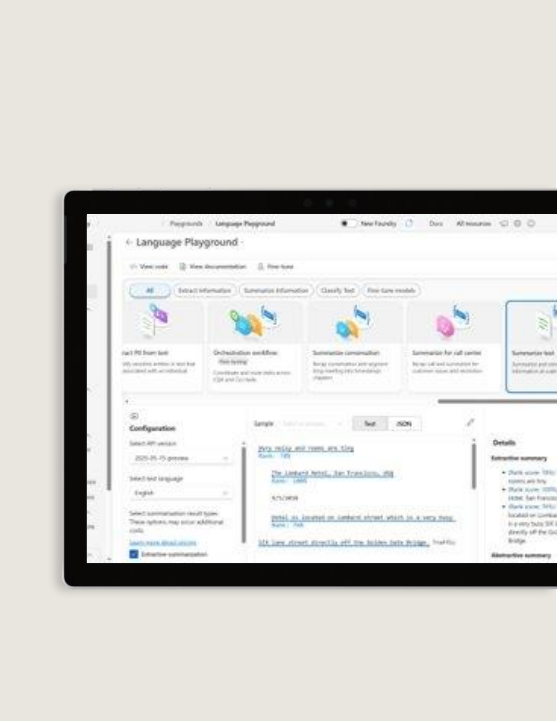
Exercise

Configure apps on Azure Kubernetes Service

In this exercise, you learn how to configure Kubernetes deployments with ConfigMaps for non-sensitive settings, Secrets for sensitive credentials, and PersistentVolumeClaims for persistent storage.

Estimated time: 30 minutes

Start the exercise at:
https://go.microsoft.com/fwlink/?linkid=2346339



© Copyright Microsoft Corporation. All rights reserved.

57

Module 3: Monitor and troubleshoot applications on Azure Kubernetes Service

© Copyright Microsoft Corporation. All rights reserved.

58

Monitor application logs and metrics

Key signals

- Response latency, error rates, and pod restart counts
- CPU and memory versus configured requests and limits
- Container exit codes and health probe results

Observation tools

- Portal Monitoring tab and Container insights for cluster-wide views
- Live Logs to stream container output in real time
- kubectl logs and kubectl top for targeted CLI queries

Command examples

```
kubectl get pods -n ai-workloads
kubectl logs <pod-name> -n ai-workloads
kubectl logs -f <pod-name> -n ai-workloads
kubectl top pods -n ai-workloads
```

© Copyright Microsoft Corporation. All rights reserved.

59

Troubleshoot pods and services

Recognize failure states

- ImagePullBackOff: registry path or pull credentials
- CrashLoopBackOff: application exits or fails health checks
- Pending: resource requests exceed available node capacity

Investigation steps

- Describe the pod and scan the Events section for clues
- Check probes, environment variables, and mounted Secrets
- Use portal Diagnose and solve problems for guided fixes

1. **Check pod status**
2. **Describe pod**
3. **Review logs**
4. **Debug container**
5. **Validate Service**

© Copyright Microsoft Corporation. All rights reserved.

60

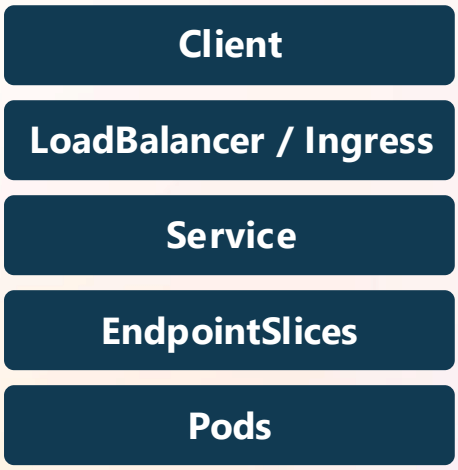
Verify service connectivity and endpoints

Service exposure

- ClusterIP for internal traffic, LoadBalancer for external access
- Selector labels must match pod labels to create endpoints
- Ports and targetPorts must align with container ports

Validation checks

- Inspect EndpointSlices to confirm pods back the Service
- Port-forward to test endpoints before external exposure
- Review Services and ingresses in the portal for external IPs



© Copyright Microsoft Corporation. All rights reserved.

61

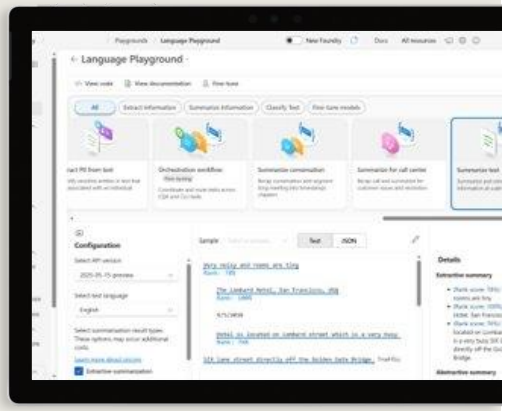
Exercise

Troubleshoot apps on Azure Kubernetes Service

In this exercise, you deploy a containerized API to Azure Kubernetes Service (AKS) and then diagnose and resolve common Kubernetes issues.

Estimated time: 30 minutes

Start the exercise at:
https://go.microsoft.com/fwlink/?linkid=2346293



© Copyright Microsoft Corporation. All rights reserved.

62

Microsoft

Develop AI solutions with Azure Cosmos DB for NoSQL

© Copyright Microsoft Corporation. All rights reserved.

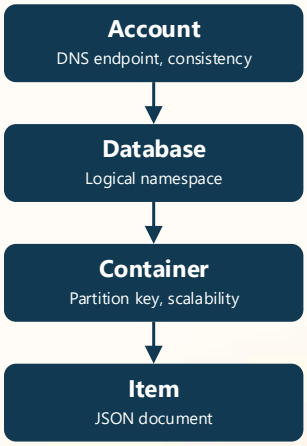
63

Module 1: Build queries for Azure Cosmos DB for NoSQL

© Copyright Microsoft Corporation. All rights reserved.

64

Explore Azure Cosmos DB for NoSQL



Resource hierarchy

- Account → Database → Container → Item

Partition key

- Most critical design decision
- Choose high-cardinality fields: `userId`, `tenantId`, `categoryId`

Throughput

- Manual: fixed RU/s
- Autoscale: scales 10%–100% of max
- All operations measured in Request Units (RUs). Point read ~1 RU, write ~5–10 RUs

© Copyright Microsoft Corporation. All rights reserved.

65

Implement the Azure Cosmos DB for NoSQL SDK

Singleton client pattern

- Reuse one `CosmosClient` for the app lifecycle
- Client maintains connection pools, caches routing
- `get_database_client()` and `get_container_client()` are lightweight handles
- `create_*_if_not_exists` for idempotent setup

	Point read	Query
When	Know id + partition key	Need to filter/search
Cost	~1 RU (1 KB)	3–hundreds of RUs
Latency	Lowest	Varies
Method	read_item()	query_items()

© Copyright Microsoft Corporation. All rights reserved.

66

Query Azure Cosmos DB for NoSQL (1 of 2)

SQL syntax and filtering

```
SELECT p.id, p.name, p.price
FROM products p
WHERE p.categoryId = "electronics"
  AND p.price < 500

-- String functions
SELECT * FROM products p
WHERE CONTAINS(p.name, "Speaker")
```

Parameterized queries

```
query = """
    SELECT * FROM products p
    WHERE p.categoryId = @category
      AND p.price < @maxPrice
"""
parameters = [
    {"name": "@category",
     "value": "electronics"},
    {"name": "@maxPrice",
     "value": 500.00}
]
items = container.query_items(
    query=query,
    parameters=parameters,
    partition_key="electronics"
)
```

© Copyright Microsoft Corporation. All rights reserved.

67

Query Azure Cosmos DB for NoSQL (2 of 2)

Single-partition (preferred)

```
items = container.query_items(
    query=query,
    parameters=params,
    partition_key="electronics"
)
```

- Routes to one partition
- Lower RU cost, lower latency
- Specify partition_key param or include in WHERE

Cross-partition

```
items = container.query_items(
    query=query,
    parameters=params,
    enable_cross_partition_query=True
)
```

- Fans out to all partitions
- Higher RU cost
- Needed when filter doesn't include partition key

© Copyright Microsoft Corporation. All rights reserved.

68

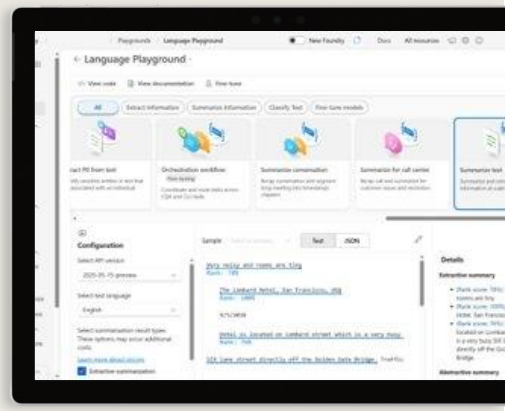
Exercise

Build a RAG document store on Azure Cosmos DB for NoSQL

In this exercise, you create an Azure Cosmos DB for NoSQL database that serves as a document store for retrieval-augmented generation (RAG) applications.

Estimated time: 30 minutes

Start the exercise at:
https://go.microsoft.com/fwlink/?linkid=2350700



© Copyright Microsoft Corporation. All rights reserved.

69

Module 2: Implement vector search on Azure Cosmos DB for NoSQL

© Copyright Microsoft Corporation. All rights reserved.

70

Store and retrieve embeddings in Azure Cosmos DB

Vector policy (set at container creation)

- **path:** JSON path to embedding property
- **dataType:** `float32`, `float16`, `int8`, `uint8`
- **dimensions:** must match your model (1536 for ada-002)
- **distanceFunction:** cosine, dotproduct, euclidean

Index types

- **flat:** exact search, max 505 dimensions
- **quantizedFlat:** compressed, up to ~50K vectors/partition
- **diskANN:** best for large scale, up to 4096 dimensions

Document structure

```
{
  "id": "doc-123",
  "title": "WiFi troubleshooting",
  "content": "Common WiFi issues...",
  "category": "networking",
  "embedding": [0.023, -0.041, ...]
}
```

Key decisions

- Exclude `/embedding/*` from standard range indexes
- Vector policies can't be modified after creation
- `float16` saves 50% storage with minimal quality loss

© Copyright Microsoft Corporation. All rights reserved.

71

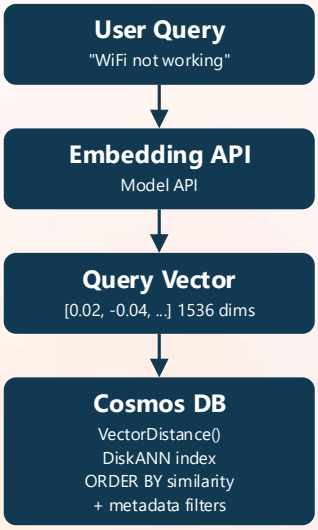
Execute vector similarity queries for semantic search

Query pattern

```
SELECT TOP 10
    c.title,
    VectorDistance(c.embedding,
        @queryVector) AS Score
FROM c
ORDER BY VectorDistance(
    c.embedding, @queryVector)
```

Essentials

- Same embedding model for queries and documents
- `TOP N` is required. Omitting it scans everything
- Parameterize with `@queryVector` for plan caching
- Scores: +1 identical, 0.7-0.9 highly similar



© Copyright Microsoft Corporation. All rights reserved.

72

Combine vector similarity results with metadata filtering

Filtered vector search

```
SELECT TOP 10 c.title,
    VectorDistance(c.embedding,
        @queryVector) AS Score
FROM c
WHERE c.category = @category
    AND c.createdDate > @startDate
ORDER BY VectorDistance(
    c.embedding, @queryVector)
```

- Pre-filtering reduces search space
- Partition key in WHERE enables single-partition routing
- Pass `partition_key` parameter for maximum efficiency

Hybrid search with RRF

```
SELECT TOP 10 *
FROM c
ORDER BY RANK RRF(
    VectorDistance(c.embedding,
        @queryVector),
    FullTextScore(c.content,
        @searchTerms)
)
```

- RRF merges semantic + keyword rankings
- Requires full-text policy and full-text index
- Weights adjust influence: `[2, 1]` favors vector
- Ideal for error codes, product names + natural language

© Copyright Microsoft Corporation. All rights reserved.

73

Use the change feed to trigger embedding refresh

Push model (Azure Functions)

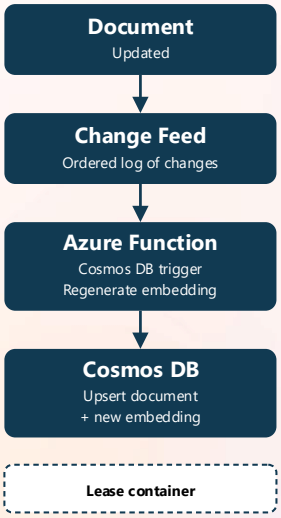
- Cosmos DB trigger auto-delivers changes
- Lease container tracks partitions and checkpoints
- Handles failover and horizontal scaling

Selective refresh

- Hash content fields to detect meaningful changes
- Metadata-only updates skip embedding regeneration
- Batch embedding API calls for high-volume scenarios

Error handling

- Processing must be idempotent
- Handle deleted documents gracefully
- Use dead-letter queues for persistent failures



© Copyright Microsoft Corporation. All rights reserved.

74

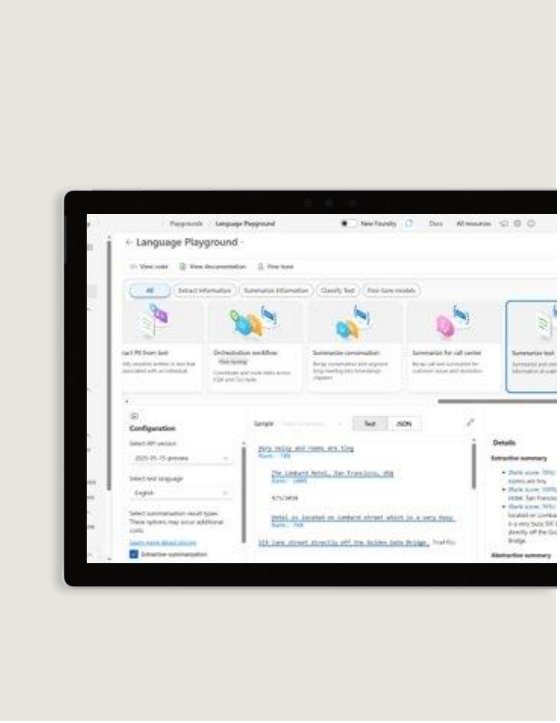
Exercise

Build a semantic search application with Azure Cosmos DB for NoSQL

In this exercise, you implement vector similarity search using Azure Cosmos DB for NoSQL.

Estimated time: 30 minutes

Start the exercise at:
https://go.microsoft.com/fwlink/?linkid=2350601



© Copyright Microsoft Corporation. All rights reserved.

75

Module 3: Optimize query performance for Azure Cosmos DB for NoSQL

© Copyright Microsoft Corporation. All rights reserved.

76

Understand indexes in Azure Cosmos DB

Index types

- Range: equality, range filters, single-property ORDER BY
- Composite: multi-property sort and combined filter + sort
- Vector: VectorDistance similarity search (flat, quantizedFlat, diskANN)
- Spatial: geospatial queries (ST_DISTANCE, ST_WITHIN)

Path syntax

- `/*` indexes all properties recursively
- `/property/?` indexes a scalar value
- `/array/[]` indexes array elements
- More specific paths override general ones

© Copyright Microsoft Corporation. All rights reserved.

77

Configure range and composite indexes

Range indexes for filters

```
SELECT * FROM c
WHERE c.documentType = 'pdf'
  AND c.uploadDate > '2024-01-01'
```

Composite indexes for sorting

```
SELECT * FROM c
ORDER BY c.relevanceScore DESC,
         c.uploadDate DESC
```

Requires a matching composite index:

```
[
  { "path": "/relevanceScore",
    "order": "descending" },
  { "path": "/uploadDate",
    "order": "descending" }
]
```

Filter + sort optimization

Include filter property in ORDER BY to use one composite index:

```
SELECT * FROM c
WHERE c.documentType = 'pdf'
ORDER BY c.documentType,
         c.uploadDate DESC
```

Composite index rules

- Property sequence must match ORDER BY exactly
- Supports reversed order on all paths
- Mixed orders need separate indexes
- Equality filters first, then one range filter last
- Each composite index adds write overhead

© Copyright Microsoft Corporation. All rights reserved.

78

Tune vector indexes for embedding workloads

Type	Best for	Max dims
flat	<1K vectors, exact	505
quantizedFlat	1K-50K vectors	4,096
diskANN	>50K vectors	4,096

Key decisions

- quantizedFlat and diskANN need 1,000+ vectors to activate
- Below threshold: falls back to brute-force scan
- DiskANN recommended for production AI apps
- Exclude embeddings from range indexes
- Vector policies are immutable after container creation
- float16 saves 50% storage vs float32
- Pre-filtering with metadata indexes reduces vector search RU cost

© Copyright Microsoft Corporation. All rights reserved.

79

Reduce RU costs with strategic indexing

Analyze before indexing

- Catalog filter properties (WHERE clauses)
- Catalog sort properties (ORDER BY)
- Identify property combinations in queries
- Prioritize by query frequency

Query metrics to monitor

- Index utilization percentage
- Retrieved vs. output document count
- RU charge per query (x-ms-request-charge)

Selective indexing strategy

- Exclude all by default: `"excludedPaths": [{"path": "/*"}]`
- Include only queried properties
- Don't index: large text, summaries, raw metadata
- Always include partition key path
- Focus composite indexes on top 5-10 query patterns

Workload profile

- Write-heavy: minimize indexes, accept higher query cost
- Read-heavy: invest in indexes, most AI apps are here
- Each composite index increases write RU cost

© Copyright Microsoft Corporation. All rights reserved.

80

Choose consistency levels for optimal performance

Level	Replicas	RU cost
Strong	2	2x
Bounded staleness	2	2x
Session	1	1x
Consistent prefix	1	1x
Eventual	1	1x

Guidance for AI apps

- Session: most AI apps, read-your-writes guarantee
- Eventual: analytics, background processing, lowest cost
- Strong: financial or regulatory scenarios only, 2x RU cost
- Pass session tokens from writes to subsequent reads
- Create separate client instances per consistency level
- Vector search tolerates eventual consistency in most cases

© Copyright Microsoft Corporation. All rights reserved.

81

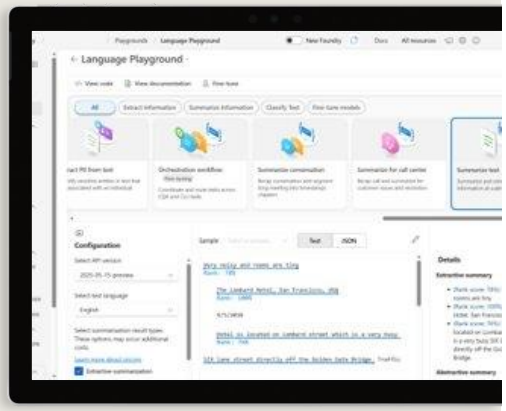
Exercise

Optimize query performance with vector indexes on Azure Cosmos DB for NoSQL

In this exercise, you implement vector similarity search using Azure Cosmos DB for NoSQL.

Estimated time: 30 minutes

Start the exercise at:
https://go.microsoft.com/fwlink/?linkid=2350800



© Copyright Microsoft Corporation. All rights reserved.

82

Microsoft

Develop AI solutions with Azure Database for PostgreSQL

© Copyright Microsoft Corporation. All rights reserved.

83

Module 1: Build and query with Azure Database for PostgreSQL

© Copyright Microsoft Corporation. All rights reserved.

84

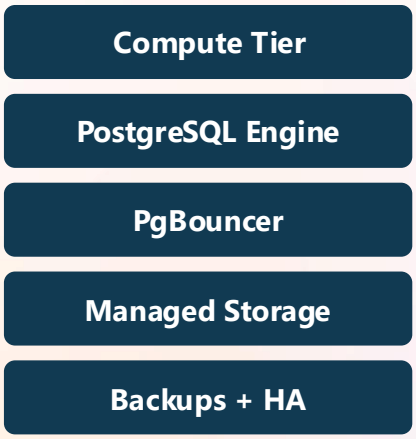
Explore Azure Database for PostgreSQL

Compute tiers:

- Burstable: dev/test, intermittent workloads
- General Purpose: production, steady performance
- Memory Optimized: large in-memory datasets, analytics

Managed capabilities:

- Automated backups with point-in-time restore (up to 35 days)
- Built-in PgBouncer on General Purpose and Memory Optimized tiers
- Extensions: pgvector, pg_trgm, hstore



© Copyright Microsoft Corporation. All rights reserved.

85

Connect to PostgreSQL

Authentication options:

- Microsoft Entra ID: token-based, no stored passwords
- PostgreSQL native: username/password, store in Key Vault

TLS encryption:

- TLS 1.2+ enforced by default
- Use sslmode=verify-full in production

Network access:

- Public access with firewall rules
- Private access with VNet integration



© Copyright Microsoft Corporation. All rights reserved.

86

Create and manage schemas

Table design decisions:

- BIGSERIAL for auto-increment, UUID for external IDs
- JSONB for variable-structure metadata
- TIMESTAMPTZ for all temporal data

Constraints enforce data integrity:

- NOT NULL, CHECK, UNIQUE at the database level
- Foreign keys with CASCADE or RESTRICT actions
- Composite primary keys for junction tables

```
CREATE TABLE conversations (
  id BIGSERIAL PRIMARY KEY,
  session_id UUID NOT NULL,
  user_id VARCHAR(255) NOT NULL,
  started_at TIMESTAMPTZ
    DEFAULT CURRENT_TIMESTAMP,
  metadata JSONB DEFAULT '{}'
);

CREATE TABLE messages (
  id BIGSERIAL PRIMARY KEY,
  conversation_id BIGINT NOT NULL
    REFERENCES conversations(id),
  role VARCHAR(50) NOT NULL,
  content TEXT NOT NULL,
  created_at TIMESTAMPTZ
    DEFAULT CURRENT_TIMESTAMP
);
```

© Copyright Microsoft Corporation. All rights reserved.

87

Query data

PostgreSQL-specific features:

- JSONB operators: `->`, `->>`, `@>`, `?`
- RETURNING clause on INSERT/UPDATE/DELETE
- ON CONFLICT for upsert operations

Performance patterns:

- Keyset pagination over OFFSET for large datasets
- CTEs for readable multi-step queries
- Recursive CTEs for hierarchical data

```
-- Upsert with ON CONFLICT
INSERT INTO user_preferences
  (user_id, key, value)
VALUES ('u1', 'theme', 'dark')
ON CONFLICT (user_id, key)
DO UPDATE SET
  value = EXCLUDED.value,
  updated_at = CURRENT_TIMESTAMP;

-- Keyset pagination
SELECT id, content, created_at
FROM messages
WHERE (created_at, id)
  < (:last_ts, :last_id)
ORDER BY created_at DESC, id DESC
LIMIT 20;
```

© Copyright Microsoft Corporation. All rights reserved.

88

Integrate SDKs and applications

psycopg 3 for Python:

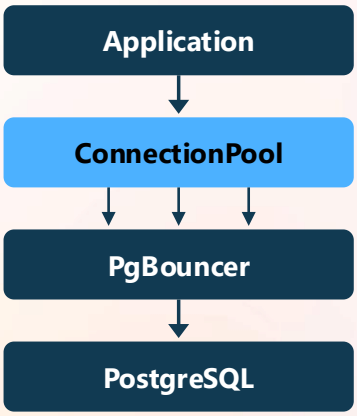
- Parameterized queries with %s placeholders
- Context managers for automatic cleanup

Connection management:

- ConnectionPool for reusable connections
- Timeouts and retry with exponential backoff

Batch performance:

- executemany for hundreds of rows
- COPY command for 10,000+ rows



© Copyright Microsoft Corporation. All rights reserved.

89

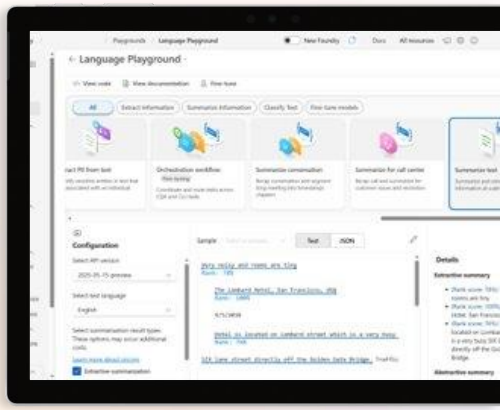
Exercise

Build an agent tool backend on Azure Database for PostgreSQL

In this exercise, you create an Azure Database for PostgreSQL instance that serves as a tool backend for an AI agent. The database stores conversation context and task state that an agent can read and write during operation.

Estimated time: 30 minutes

Start the exercise at:
https://go.microsoft.com/fwlink/?linkid=2349114



© Copyright Microsoft Corporation. All rights reserved.

90

Module 2: Implement vector search with Azure Database for PostgreSQL

© Copyright Microsoft Corporation. All rights reserved.

91

Store and query embeddings with pgvector

Schema design:

- Define columns with `vector(n)` matching your model's output dimensions
- Store embeddings in the same row as metadata for single-query retrieval

Distance operators:

- `<->` L2 (Euclidean) distance
- `<=>` Cosine distance, most common for text embeddings
- `<#>` Negative inner product

```
CREATE TABLE documents (
    id SERIAL PRIMARY KEY,
    title TEXT NOT NULL,
    content TEXT,
    category TEXT,
    embedding vector(1536)
);
```

© Copyright Microsoft Corporation. All rights reserved.

92

Perform fast vector similarity search

IVFFlat

- Clusters vectors into partitioned lists
- Requires existing data for creation
- Faster builds, lower memory

```
USING ivfflat (embedding vector_cosine_ops)
WITH (lists = 100);
```

HNSW

- Multi-layer navigable graph structure
- Indexes empty tables incrementally
- Higher recall and query performance

```
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);
```

© Copyright Microsoft Corporation. All rights reserved.

93

Manage index lifecycle and embedding updates

Index maintenance:

- Monitor with pg_stat_user_indexes for usage and size
- Rebuild when data distribution shifts significantly
- Use CREATE INDEX CONCURRENTLY for zero-downtime rebuilds

Embedding updates:

- Batch updates in 1,000-5,000 row transactions
- Use a stale flag column for async refresh
- Parallel column strategy for model migrations

1. Add new vector column
2. Create index on new column
3. Backfill embeddings in batches
4. Validate search quality
5. Drop old column and index

© Copyright Microsoft Corporation. All rights reserved.

94

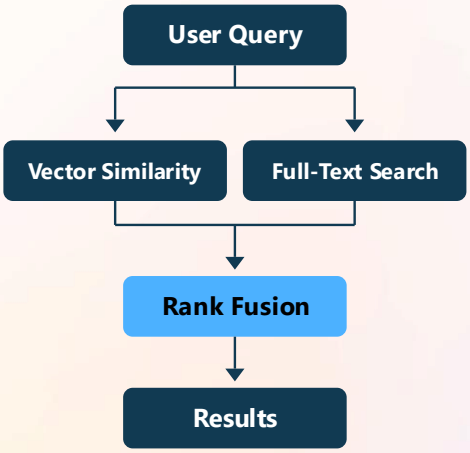
Run vector similarity search for semantic retrieval

Refining results:

- Combine vector similarity with WHERE clauses on metadata
- Set distance thresholds to prevent low-relevance results
- Add B-tree indexes on frequently filtered columns

Hybrid search:

- Combine vector similarity with full-text search for lexical matches
- Reciprocal Rank Fusion balances different scoring scales



© Copyright Microsoft Corporation. All rights reserved.

95

Implement retrieval patterns for RAG pipelines

RAG retriever design:

- Separate source_documents and document_chunks tables
- Link chunks to sources for citation metadata

Retrieval patterns:

- Fetch adjacent chunks for surrounding context
- Track cumulative token counts to stay within LLM limits
- Measure precision, recall, and MRR to evaluate quality

```
CREATE TABLE document_chunks (
    id SERIAL PRIMARY KEY,
    document_id INT REFERENCES
        source_documents(id),
    chunk_index INT NOT NULL,
    content TEXT NOT NULL,
    embedding vector(1536),
    token_count INT
);
```

© Copyright Microsoft Corporation. All rights reserved.

96

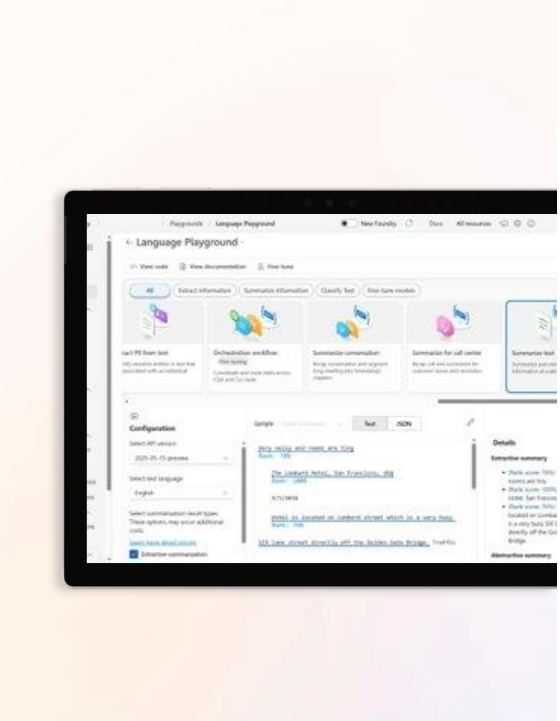
Exercise

Implement vector search on Azure Database for PostgreSQL

In this exercise, you build a product similarity search application using Azure Database for PostgreSQL and the pgvector extension.

Estimated time: 30 minutes

Start the exercise at:
https://go.microsoft.com/fwlink/?linkid=2348708



© Copyright Microsoft Corporation. All rights reserved.

97

Module 3: Optimize vector search in Azure Database for PostgreSQL

© Copyright Microsoft Corporation. All rights reserved.

98

Introduction

- pgvector extension enables vector similarity search in PostgreSQL
- Supports IVFFlat and HNSW approximate nearest neighbor indexes
- Azure Database for PostgreSQL provides managed infrastructure with built-in PgBouncer
- Configuration tuning, indexing, and connection management directly affect query latency
- Optimizations support recommendations, semantic search, and RAG pipelines



© Copyright Microsoft Corporation. All rights reserved.

99

Tune PostgreSQL for pgvector

Memory and planner tuning:

- `shared_buffers` and `work_mem` sized for vector data volume
- `random_page_cost` lowered to 1.1 for SSD storage
- `effective_io_concurrency` set to 200 for parallel I/O

pgvector search parameters:

- `ivfflat.probes` controls recall vs speed trade-off
- `hnsw.ef_search` controls graph exploration depth
- `EXPLAIN ANALYZE` validates index usage and timing

```
-- Server parameters
SET shared_buffers = '2GB';
SET work_mem = '256MB';
SET random_page_cost = 1.1;
SET effective_io_concurrency = 200;

-- Search parameters (per session)
SET ivfflat.probes = 10;
SET hnsw.ef_search = 100;

-- Validate index usage
EXPLAIN ANALYZE
SELECT * FROM products
ORDER BY embedding <=> $1 LIMIT 10;
```

© Copyright Microsoft Corporation. All rights reserved.

100

Choose and configure vector indexes

IVFFlat (cluster-based):

- Partitions vectors into lists via k-means clustering
- Fast build times, lower memory usage
- Best for frequent rebuilds or limited memory budgets

HNSW (graph-based):

- Multi-layer navigable graph structure
- Higher recall (99%+), consistent query latency
- Best for read-heavy, accuracy-critical workloads



© Copyright Microsoft Corporation. All rights reserved.

101

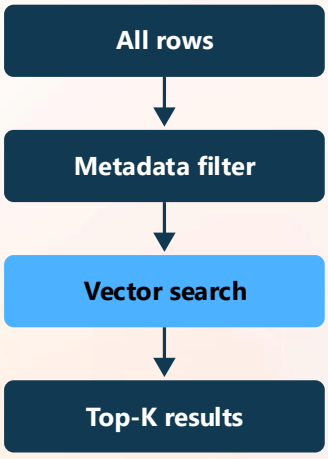
Optimize data layout

Schema design for filtered search:

- Structured columns with B-tree indexes for common filters
- JSONB for dynamic or rarely filtered attributes
- Composite indexes match multi-column filter patterns

Query execution patterns:

- Metadata filters narrow candidates before vector search
- Post-filter pattern for low-selectivity predicates
- Table partitioning for tens of millions of rows



© Copyright Microsoft Corporation. All rights reserved.

102

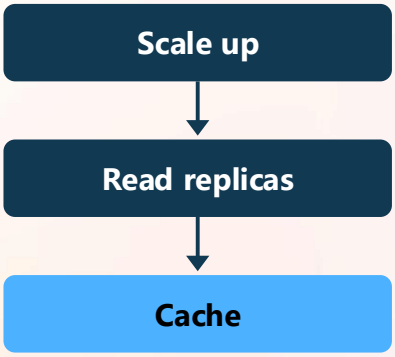
Scale for high-volume workloads

Vertical scaling:

- Memory Optimized tier (8 GB/vCore) keeps indexes cached
- Scale up when single-query latency is too high
- Monitor CPU, memory, and storage I/O thresholds

Horizontal scaling and caching:

- Up to five read replicas for read-heavy distributions
- Azure Cache for Redis for frequently requested data
- Reserved capacity discounts up to 65% for stable baselines



© Copyright Microsoft Corporation. All rights reserved.

103

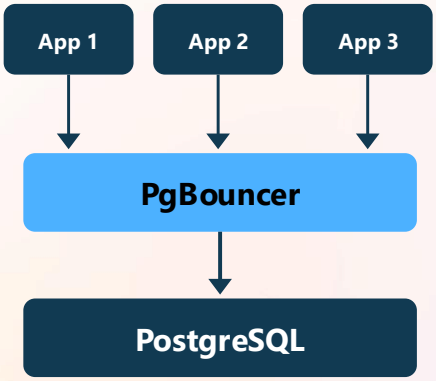
Connection optimization

PgBouncer connection pooling:

- Built-in on General Purpose and Memory Optimized tiers
- Transaction mode multiplexes connections efficiently
- Reduces 50-200ms overhead per new connection

Application-level patterns:

- Pool size balanced across application instances
- SET LOCAL for per-transaction vector search parameters
- Retry with exponential backoff for transient failures



© Copyright Microsoft Corporation. All rights reserved.

104

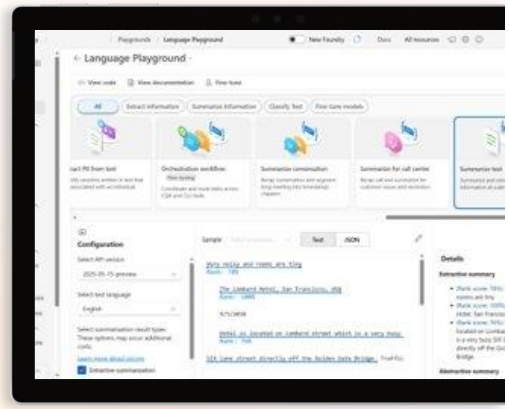
Exercise

Optimize vector search performance in Azure Database for PostgreSQL

In this exercise, you deploy an Azure Database for PostgreSQL instance and optimize it for vector search workloads.

Estimated time: 30 minutes

Start the exercise at:
https://go.microsoft.com/fwlink/?linkid=2348919



© Copyright Microsoft Corporation. All rights reserved.

105

Microsoft

Enhance AI solutions with Azure Managed Redis

© Copyright Microsoft Corporation. All rights reserved.

106

Module 1: Implement data operations in Azure Managed Redis

© Copyright Microsoft Corporation. All rights reserved.

107

Explore Azure Managed Redis

Caching strategies:

- Data cache: cache-aside pattern loads data on demand, invalidates on change
- Content cache: static content (headers, footers, templates) served from memory
- Session store: user state stored by session ID, replaces cookie-heavy approaches

Choosing a tier:

- Memory Optimized: high memory-to-vCPU (8:1), lower throughput, suits dev/test
- Balanced: standard workloads with 4:1 memory-to-vCPU ratio
- Compute Optimized: maximum throughput with 2:1 memory-to-vCPU ratio
- Flash Optimized: moves cold data from RAM to NVMe for large datasets

© Copyright Microsoft Corporation. All rights reserved.

108

Client libraries and development best practices

Client libraries:

- redis-py (Python), StackExchange.Redis (.NET), Lettuce (Java), node_redis (Node.js)
- Enterprise clustering works with all clients. OSS clustering requires cluster-aware clients

Development best practices:

- Keep key values small. Divide large data into multiple keys
- Use pipelining for batch operations
- Avoid `KEYS` in production. Use `SCAN` for non-blocking iteration
- Co-locate Redis and application in the same Azure region
- Scale when metrics exceed 75%: memory, CPU, clients, bandwidth

```
# Pipelining: single round-trip
# for multiple hash operations
pipe = r.pipeline()
pipe.hgetall('user:1001')
pipe.hgetall('user:1002')
results = pipe.execute()

# Retrieve specific fields
pipe = r.pipeline()
pipe.hget('user:1001', 'name')
pipe.hget('user:1002', 'name')
names = pipe.execute()
```

© Copyright Microsoft Corporation. All rights reserved.

109

Implement data operations (1 of 2)

Core data types:

- Strings: `SET/GET` for text or serialized data
- Hashes: field-value pairs in a single key, efficient for structured data
- Lists: ordered collections for queues or activity feeds
- Numeric: atomic `INCR/DECR` for counters and rate limiting

Connection:

- Default port for Azure Managed Redis: 10000
- Use `redis.Redis` for Enterprise clustering, `redis.cluster.RedisCluster` for OSS clustering

```
# Connect to Azure Managed Redis
r = redis.Redis(
    host='your-instance',
    port=10000,
    ssl=True,
    decode_responses=True
)

# String operations
r.set('user:1001:name', 'Alice')
name = r.get('user:1001:name')

# Hash operations
r.hset('user:1001', mapping={
    'name': 'Alice',
    'email': 'alice@example.com'
})
```

© Copyright Microsoft Corporation. All rights reserved.

110

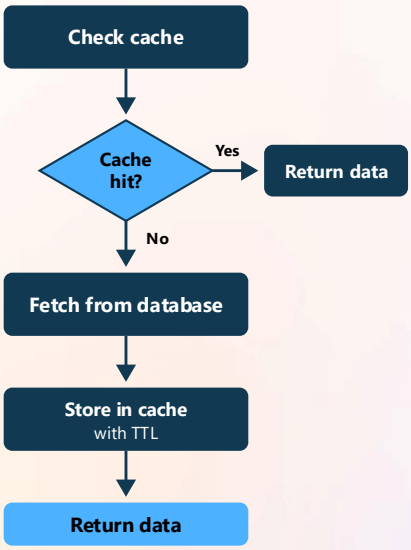
Implement data operations (2 of 2)

Expiration management:

- `SETEX` sets value and TTL atomically
- `EXPIRE` adds TTL to existing keys of any type
- TTL of -1 means no expiration set. TTL of -2 means key doesn't exist

Cache invalidation patterns:

- Time-based: automatic expiration via TTL
- Manual: delete cache keys when source data changes
- Cache-aside: check cache, fetch on miss, store with TTL
- Pattern-based: use `SCAN` to find and delete related keys by pattern



© Copyright Microsoft Corporation. All rights reserved.

111

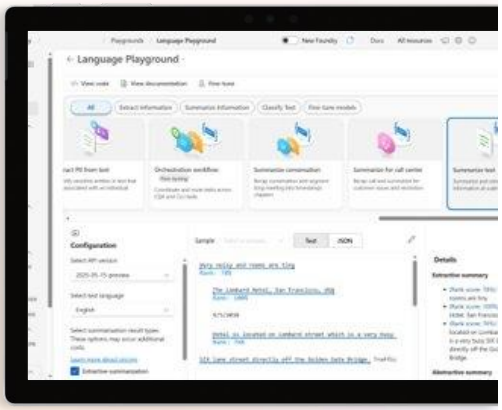
Exercise

Perform data operations in Azure Managed Redis

In this exercise, you create an Azure Managed Redis resource and build a Python console application that performs common data operations using the redis-py library.

Estimated time: 30 minutes

Start the exercise at:
https://go.microsoft.com/fwlink/?linkid=2344201



© Copyright Microsoft Corporation. All rights reserved.

112

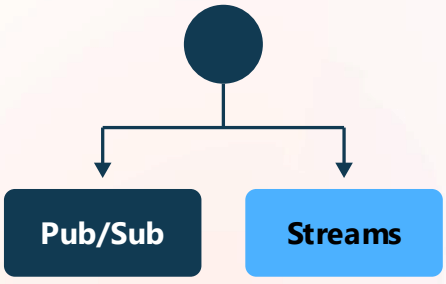
Module 2: Implement event messaging with Azure Managed Redis

© Copyright Microsoft Corporation. All rights reserved.

113

Introduction

- Redis provides two messaging patterns: pub/sub and Streams
- Pub/sub broadcasts events to all active listeners in real time
- Streams persist messages as durable task queues with consumer coordination
- AI applications use both: pub/sub for notifications, Streams for processing pipelines
- Available in both Azure Cache for Redis and Azure Managed Redis



© Copyright Microsoft Corporation. All rights reserved.

114

Publish and subscribe to events with Redis pub/sub

How pub/sub works:

- Publishers send messages to named channels, subscribers listen to channels of interest
- At-most-once delivery: only active subscribers receive messages
- Fire-and-forget: no persistence, no delivery confirmation

When to use pub/sub:

- Cache invalidation across multiple API instances
- Real-time status updates and notifications
- Configuration change broadcasts
- Event-driven coordination between loosely coupled services

```
# Publisher sends to a channel
redis_client.publish(
    'ai:models:updated',
    f'{model_name}:{version}'
)

# Subscriber listens
pubsub = redis_client.pubsub()
pubsub.subscribe('ai:models:updated')

for msg in pubsub.listen():
    if msg['type'] == 'message':
        clear_model_cache(msg['data'])

# Pattern subscription
pubsub.psubscribe('ai:*')
```

© Copyright Microsoft Corporation. All rights reserved.

115

Implement task queues with Redis Streams

What Streams provide:

- Append-only task queue with durable message storage
- Consumer groups distribute tasks across workers automatically
- Unacknowledged tasks stay pending for automatic retry
- Processing history for debugging and replay

Core commands:

- `XADD` adds a task to the Stream
- `XREADGROUP` pulls tasks for a specific consumer group
- `XACK` marks a task as complete
- `XPENDING` / `XCLAIM` handle stuck or failed tasks

```
# Add a task to the Stream
task_id = redis.xadd(
    'ai:inference:queue',
    {'prompt': '...', 'model': 'gpt-4'}
)

# Worker reads from consumer group
messages = redis.xreadgroup(
    'workers', 'worker-1',
    {'ai:inference:queue': '>'},
    count=5, block=5000
)

# Acknowledge completion
redis.xack(
    'ai:inference:queue',
    'workers', task_id
)
```

© Copyright Microsoft Corporation. All rights reserved.

116

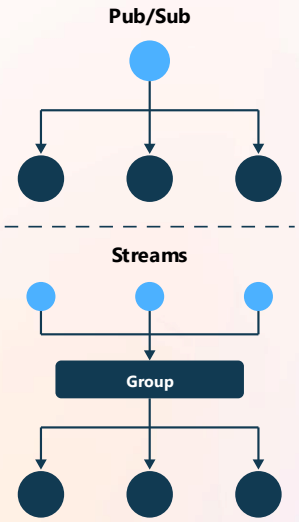
Choose between broadcast and coordinated distribution

Pub/sub (broadcast):

- Every subscriber receives every message
- No delivery guarantees or acknowledgments
- Best for: cache invalidation, notifications, config updates

Streams (coordinated):

- Each message goes to exactly one consumer in a group
- Built-in retry, acknowledgment, and failure recovery
- Best for: task queues, processing pipelines, work distribution



© Copyright Microsoft Corporation. All rights reserved.

117

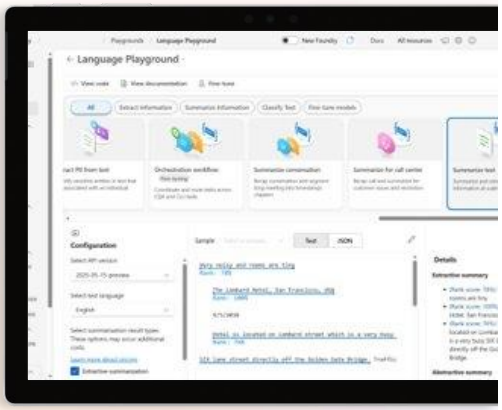
Exercise

Publish and subscribe to events in Azure Managed Redis

In this exercise, you create an Azure Managed Redis resource and complete the code for a console-based publisher and a subscriber app.

Estimated time: 30 minutes

Start the exercise at:
https://go.microsoft.com/fwlink/?linkid=2344003



© Copyright Microsoft Corporation. All rights reserved.

118

Module 3: Implement vector storage in Azure Managed Redis

© Copyright Microsoft Corporation. All rights reserved.

119

Index and query vector data (1 of 2)

Create a vector index:

- Define schema with VectorField (algorithm, type, dimensions, metric)
- Index auto-syncs as vectors are added to matching key prefixes

Ingest vectors:

- Convert embeddings to bytes with `tobytes()`
- Use pipelines for bulk ingestion (10-100x throughput)

```
schema = (
    TextField("title"),
    VectorField("embedding", "HNSW", {
        "TYPE": "FLOAT32",
        "DIM": 1536,
        "DISTANCE_METRIC": "COSINE"
    })
)

redis_client.ft("idx:docs").create_index(
    fields=schema,
    definition=IndexDefinition(
        prefix=["doc:"],
        index_type=IndexType.HASH
    )
)
```

© Copyright Microsoft Corporation. All rights reserved.

120

Index and query vector data (2 of 2)

Query types:

- KNN: fixed count of nearest neighbors
- Hybrid: metadata filter + vector similarity
- Range: all vectors within a distance threshold

Key considerations:

- Dimensions must match between index and queries
- Lower distance scores = higher similarity
- Requires `dialect(2)` for vector queries

```
# KNN: find 5 nearest neighbors
q = Query(
    "*=>[KNN 5 @embedding $vec AS score]"
).sort_by("score").dialect(2)

# Hybrid: filter then search
q = Query(
    "@category:{docs}=>"
    "[KNN 3 @embedding $vec AS score]"
).dialect(2)
```

© Copyright Microsoft Corporation. All rights reserved.

121

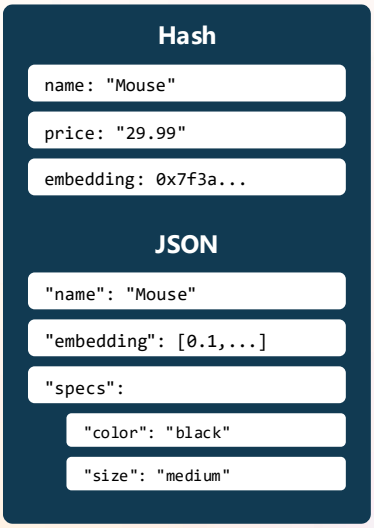
Optimize Redis data structures for vector storage

Hash storage:

- Flat field-value pairs, vectors as binary bytes
- Lower memory, faster queries
- Best for simple metadata

JSON storage:

- Nested objects and arrays supported
- Vectors as numeric arrays with `tolist()`
- Best for complex, hierarchical data



© Copyright Microsoft Corporation. All rights reserved.

122

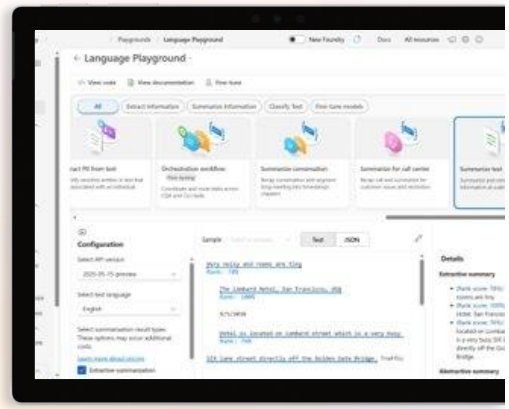
Exercise

Implement semantic search in Azure Managed Redis

In this exercise, you create an Azure Managed Redis resource and complete the code for a vector storage application.

Estimated time: 30 minutes

Start the exercise at:
https://go.microsoft.com/fwlink/?linkid=2344301



© Copyright Microsoft Corporation. All rights reserved.

123

Microsoft

Integrate Backend Services for AI Solutions

© Copyright Microsoft Corporation. All rights reserved.

124

Module 1: Queue and process AI operations with Azure Service Bus

© Copyright Microsoft Corporation. All rights reserved.

125

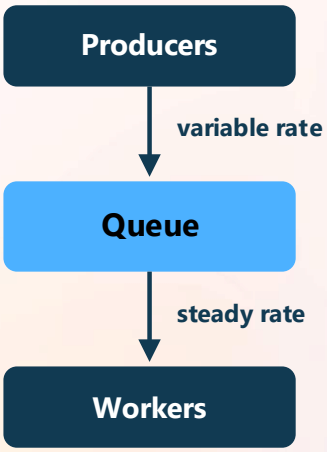
Explore Azure Service Bus concepts and messaging in AI architectures

Why AI needs messaging:

- Inference latency is unpredictable. Synchronous calls block under load
- Decouples producers (APIs) from consumers (model processors)

Core patterns:

- Load leveling: queue absorbs traffic spikes, processors consume at a steady rate
- Competing consumers: multiple workers on one queue, each message delivered once
- Temporal decoupling: producers and consumers don't need to be online simultaneously



© Copyright Microsoft Corporation. All rights reserved.

126

Choose between queues and topics with subscriptions

Queues (point-to-point):

- Each message delivered to exactly one consumer
- Competing consumers for horizontal scaling
- Sessions enable FIFO ordering per session group

Topics (fan-out):

- Message copies distributed to all subscriptions
- Each subscription acts as an independent virtual queue
- Add subscribers without modifying the producer

```
# Queue: point-to-point
sender = client.get_queue_sender("requests")
receiver = client.get_queue_receiver("requests")

# Topic: fan-out
sender = client.get_topic_sender("results")
receiver = client.get_subscription_receiver(
    topic_name="results",
    subscription_name="notifications"
)
```

© Copyright Microsoft Corporation. All rights reserved.

127

Structure messages for AI workloads (1 of 2)

Message anatomy:

- Body: serialized payload (JSON for prompts and parameters)
- Application properties: custom metadata for routing and filtering
- System properties: `message_id`, `correlation_id`, `content_type`, `time_to_live`

Correlation tracking:

- `correlation_id` tracks a request end-to-end across all pipeline stages
- `message_id` enables duplicate detection within the detection window

```
# Message with JSON payload and tracking properties
message = ServiceBusMessage(
    body=json.dumps({
        "prompt": "Extract parties...",
        "model": "gpt-4o",
        "temperature": 0.1
    }),
    content_type="application/json",
    message_id=str(uuid.uuid4()),
    correlation_id="req-abc-12345",
    application_properties={
        "model_name": "gpt-4o",
        "priority": "standard"
    }
)
```

© Copyright Microsoft Corporation. All rights reserved.

128

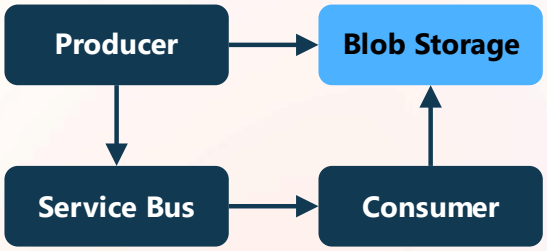
Structure messages for AI workloads (2 of 2)

Claim-check pattern:

- Upload large payloads to Blob Storage
- Send only the blob URI in the Service Bus message
- Consumer retrieves the full payload from storage
- Works within any tier's size limits

Message TTL and batching:

- `time_to_live` expires stale requests before they consume resources
- `ServiceBusMessageBatch` groups messages into a single send operation



© Copyright Microsoft Corporation. All rights reserved.

129

Process messages reliably (1 of 2)

Receive modes:

- Receive-and-delete: message removed immediately. Highest throughput, no delivery guarantee
- Peek-lock (default): message locked, consumer must settle. At-least-once delivery

Settlement operations:

- Complete: success, remove from queue
- Abandon: release lock, redeliver (transient errors)
- Dead-letter: move to DLQ with reason (permanent failures)
- Defer: keep in queue, retrieve later by sequence number

```
# Peek-lock receive with settlement
for msg in receiver:
    try:
        result = run_inference(msg)
        receiver.complete_message(msg)
    except json.JSONDecodeError:
        receiver.dead_letter_message(
            msg, reason="MalformedPayload"
        )
    except TransientServiceError:
        receiver.abandon_message(msg)
```

© Copyright Microsoft Corporation. All rights reserved.

130

Process messages reliably (2 of 2)

Dead-letter queue (DLQ):

- Automatic subqueue on every queue and subscription
- Holds expired messages, max delivery count exceeded, and explicitly dead-lettered
- Each message carries `dead_letter_reason` and `dead_letter_error_description`

Lock management:

- Default lock: 1 minute, max: 5 minutes
- `AutoLockRenewer` extends locks for long-running inference
- Two-phase processing avoids lock management for very long operations

```
# Inspect the dead-letter queue
with client.get_queue_receiver(
    queue_name="inference-requests",
    sub_queue=ServiceBusSubQueue.DEAD_LETTER,
    max_wait_time=10
) as dlq_receiver:
    for msg in dlq_receiver:
        print(msg.dead_letter_reason)
        print(msg.correlation_id)
        dlq_receiver.complete_message(msg)
```

© Copyright Microsoft Corporation. All rights reserved.

131

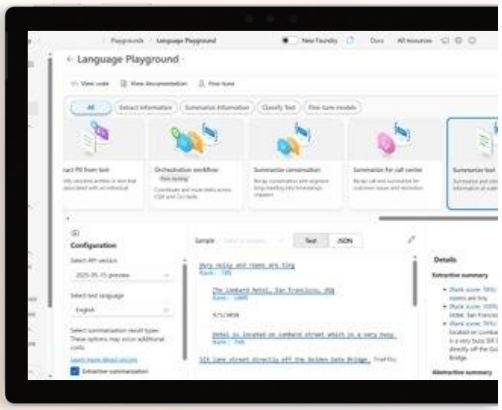
Exercise

Process messages with Azure Service Bus

In this exercise, you create an Azure Service Bus namespace and build a Python Flask web application that demonstrates core messaging patterns using an AI inference scenario.

Estimated time: 30 minutes

Start the exercise at:
https://go.microsoft.com/fwlink/?linkid=2352823



© Copyright Microsoft Corporation. All rights reserved.

132

Module 2: Develop event-driven AI workflows with Azure Event Grid

© Copyright Microsoft Corporation. All rights reserved.

133

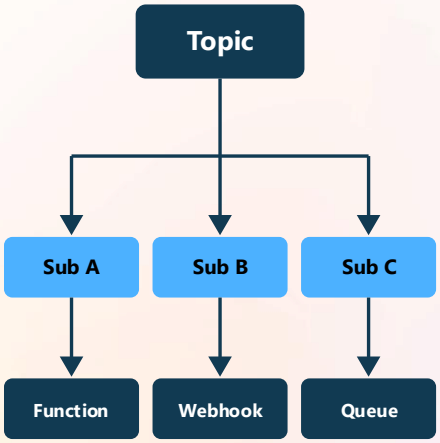
Understand Azure Event Grid concepts and event-driven patterns for AI solutions

Core components:

- Events notify of state changes; sources emit them to topics
- Topics receive events; subscriptions filter and route them
- Handlers process: Functions, webhooks, Service Bus, queues

Event-driven benefits for AI:

- Replaces polling with sub-second reactive triggers
- Independent scaling and fault isolation across pipeline stages



© Copyright Microsoft Corporation. All rights reserved.

134

Work with event schemas and properties

CloudEvents v1.0 schema:

- CNCF standard, recommended for new implementations
- Required: `specversion`, `type`, `source`, `id`
- `subject` enables hierarchical path-based filtering

Filtering options:

- Type filtering: exact match on event type
- Subject filtering: prefix/suffix path matching
- Advanced filtering: operators on data payload fields

CloudEvent for an AI inference completion:

```
{
  "specversion": "1.0",
  "type": "com.contoso.ai.InferenceCompleted",
  "source": "/services/content-moderation",
  "id": "a1b2c3d4-e5f6-7890",
  "subject": "/pipelines/moderation/batch-42",
  "data": {
    "modelName": "content-classifier-v3",
    "resultLocation": "https://.../results.json",
    "status": "success"
  }
}
```

© Copyright Microsoft Corporation. All rights reserved.

135

Configure delivery and retry policies for reliable event processing

Delivery behavior:

- HTTP POST, success = 200-204 within 30 seconds
- At-least-once delivery requires idempotent handlers
- Exponential backoff: 10s, 30s, 1m... up to 24h

Configuration options:

- Max attempts (1-30) and event TTL (1-1440 min)
- Dead-letter to Blob Storage for undeliverable events
- Output batching for high-throughput workloads



© Copyright Microsoft Corporation. All rights reserved.

136

Publish custom events from AI applications

Custom topics:

- Create with `cloudeventschemav1_0` input schema
- Authenticate with Entra ID and managed identity

Event design principles:

- Include correlation IDs for distributed tracing
- Reference results via storage URLs, keep payloads small
- Publish at natural pipeline checkpoints

```python
# Publish a CloudEvent using the Event Grid SDK
from azure.core.messaging import CloudEvent
from azure.eventgrid import EventGridPublisherClient

client = EventGridPublisherClient(
    endpoint, credential
)

event = CloudEvent(
    type="com.contoso.ai.InferenceCompleted",
    source="/services/content-moderation",
    data={
        "requestId": "req-78901",
        "modelName": "classifier-v3",
        "status": "completed"
    },
    subject="/pipelines/moderation/classifier"
)
```

© Copyright Microsoft Corporation. All rights reserved.

137

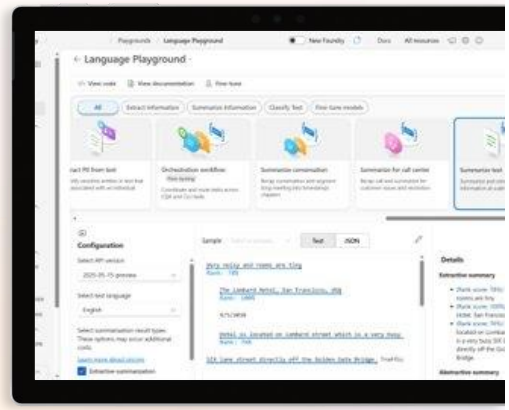
Exercise

Publish and receive events with Azure Event Grid

In this exercise, you deploy an Event Grid Namespace with a namespace topic and filtered event subscriptions, then build a Python Flask application that publishes content moderation events and receives them using pull delivery.

Estimated time: 30 minutes

Start the exercise at:
https://go.microsoft.com/fwlink/?linkid=2352234



© Copyright Microsoft Corporation. All rights reserved.

138

Module 3: Build serverless AI backends with Azure Functions

© Copyright Microsoft Corporation. All rights reserved.

139

Understand Azure Functions hosting and scaling for AI workloads

Cold start:

- Delay when a new instance is allocated and dependencies are loaded
- AI workloads amplify it: large libraries, model loading, connection warmup

Flex Consumption vs Premium:

- Flex Consumption: per-function scaling, always-ready instances, scales to zero
- Premium: pre-warmed workers, no cold starts, custom container support

Scaling and concurrency:

- Each trigger type scales independently on its own instances
- Lower concurrency isolates resources per invocation
- Instance memory determines CPU: 2,048 MB = one vCPU, 4,096 MB = two vCPU

Long-running tasks:

- HTTP triggers face a 230-second load balancer timeout
- Async request-reply: accept request, queue work, return 202
- Queue-triggered processor runs with no timeout constraint

© Copyright Microsoft Corporation. All rights reserved.

140

Set up the local development environment for Functions

Core Tools and project structure:

- Core Tools provides the local Functions runtime
- Python v2: single `function_app.py` with decorator-based definitions
- `host.json` for runtime config, `local.settings.json` for local settings

Local development workflow:

- Azurite emulates storage locally, required even for non-storage triggers
- F5 in VS Code launches Core Tools with debugger attached
- Sync settings with Azure: `func azure functionapp fetch-app-settings`

© Copyright Microsoft Corporation. All rights reserved.

141

Create triggers and bindings for AI integration patterns (1 of 2)

HTTP triggers for inference:

- `@app.route()` decorator with route, methods, and auth level
- Returns response directly to the caller
- 230-second timeout from Azure Load Balancer

Service Bus triggers for batch processing:

- `@app.service_bus_queue_trigger()` picks up queued work items
- `maxConcurrentCalls` in host.json controls parallelism per instance
- Dead-letter sub-queue catches repeatedly failing messages

```python
# HTTP trigger - inference endpoint
@app.route(
    route="classify",
    methods=["POST"],
    auth_level=func.AuthLevel.FUNCTION
)
def classify_document(req):
    payload = req.get_json()
    result = perform_classification(
        payload.get("text")
    )
    return func.HttpResponse(
        json.dumps(
            {"category": result}
        ),
        status_code=200,
        mimetype="application/json"
    )
```

© Copyright Microsoft Corporation. All rights reserved.

142

Create triggers and bindings for AI integration patterns (2 of 2)

Output bindings:

- Write results to storage without SDK boilerplate
- Runtime handles connection, auth, and serialization
- Targets: Blob Storage, Cosmos DB, Service Bus

SDK clients for AI services:

- Use for services without bindings: Document Intelligence, OpenAI, AI Search
- Initialize clients at module level for reuse across invocations
- `DefaultAzureCredential` works locally and in production

```python
# Output binding - store results
@app.service_bus_queue_trigger(
    arg_name="msg",
    queue_name="document-jobs",
    connection="ServiceBusConnection"
)
@app.blob_output(
    arg_name="output_blob",
    path="results/{rand-guid}.json",
    connection="AzureWebJobsStorage"
)
def process_and_store(msg, output_blob):
    job = json.loads(
        msg.get_body().decode("utf-8")
    )
    result = extract_and_classify(
        job["document_url"]
    )
    output_blob.set(json.dumps(result))
```

© Copyright Microsoft Corporation. All rights reserved.

143

Manage secrets and configuration in Functions

Application settings and Key Vault references:

- App settings exposed as environment variables via `os.environ`
- Key Vault references resolve secrets through app settings at runtime
- Versionless URIs auto-resolve to latest version on rotation

Azure App Configuration:

- Centralized store for non-secret config shared across services
- Label-based filtering per environment

© Copyright Microsoft Corporation. All rights reserved.

144

Configure identity and access for Functions

Identity-based connections:

- Replace connection strings with managed identity and endpoint config
- Suffix pattern: `ServiceBusConnection__fullyQualifiedNamespace`
- Assign minimum required roles per service

Authorization keys:

- Function keys scoped to one function, host keys to all functions
- System keys for extensions, master key for admin access
- Set per-trigger auth level: `anonymous`, `function`, `admin`

1. **Enable managed identity**
2. **Assign role on target resource**
3. **Set endpoint in app settings**
4. **Runtime authenticates automatically**

© Copyright Microsoft Corporation. All rights reserved.

145

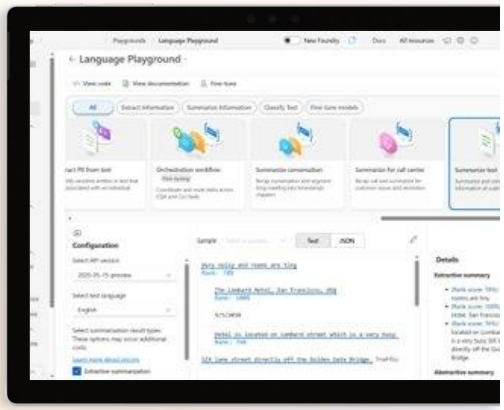
Exercise

Create an MCP server with Azure Functions

In this exercise, you create an Azure Functions project with the MCP extension, define tool trigger functions for document processing, configure the MCP server settings, and test the server locally by connecting to it from GitHub Copilot in agent mode.

Estimated time: 30 minutes

Start the exercise at:
https://go.microsoft.com/fwlink/?linkid=2352235



© Copyright Microsoft Corporation. All rights reserved.

146

Microsoft

Manage application secrets and configuration for AI solutions

© Copyright Microsoft Corporation. All rights reserved.

147

Module 1: Manage application secrets with Azure Key Vault

© Copyright Microsoft Corporation. All rights reserved.

148

Store and organize secrets, keys, and certificates

Three object types:

- Secrets: arbitrary strings like API keys, connection strings, and passwords (up to 25 KB)
- Keys: server-side cryptography operations, key material never leaves the vault
- Certificates: X.509 lifecycle management with certificate authority integration

Vault organization and access control:

- Separate vault per application and environment limits blast radius
- Secrets User role for app identities (read-only), Secrets Officer for operators
- Soft delete enabled by default, purge protection recommended for production

© Copyright Microsoft Corporation. All rights reserved.

149

Retrieve secrets using Azure SDK client libraries

SecretClient operations:

- Initialized with vault URL and credential object
- `get_secret()` returns value plus metadata: version, created date, content type
- `list_properties_of_secrets()` discovers secrets without reading values

Error handling decisions:

- ResourceNotFoundError: configuration error, don't retry
- HttpResponseError: auth or server failure, check RBAC assignments
- ServiceRequestError: transient network issue, safe to retry

```
# Retrieve a secret and access metadata
secret = client.get_secret("openai-api-key")

api_key = secret.value
version = secret.properties.version
created = secret.properties.created_on
```

© Copyright Microsoft Corporation. All rights reserved.

150

Handle secret versioning and rotation

Versioning and rotation:

- Each `set_secret()` creates a new version with a unique identifier
- `get_secret()` returns the latest enabled version by default
- Three strategies: manual, automated (Event Grid), dual-credential

Zero-downtime credential updates:

- App catches auth failure, refetches secret from vault (gets latest version)
- Retries with new credential, no restart needed
- Set `expires_on` plus Event Grid to automate rotation triggers

```
# Versioning: each set creates a new version
v1 = client.set_secret("db-key", "old-value")
v2 = client.set_secret("db-key", "new-value")

# Returns latest version (v2)
latest = client.get_secret("db-key")

# Retrieve a specific version
old = client.get_secret(
    "db-key", version=v1.properties.version
)
```

© Copyright Microsoft Corporation. All rights reserved.

151

Implement caching strategies to reduce Key Vault calls

Why cache:

- Key Vault throttles at 4,000 GET per vault per 10-second window
- Time-based cache stores secret with TTL, refreshes periodically
- Event Grid invalidation provides near-real-time freshness

Cache scope and TTL guidance:

- Per-process cache: simplest approach, recommended starting point
- Shared distributed cache: reduces total calls but adds complexity
- Match TTL to rotation frequency (90-day rotation = 1-hour TTL)

```
# Time-based cache with configurable TTL
_cache = {}
_ttl = 900  # 15 minutes

def get_cached_secret(client, name):
    cached = _cache.get(name)
    now = time.monotonic()
    if cached and (now - cached["ts"]) < _ttl:
        return cached["value"]
    secret = client.get_secret(name)
    _cache[name] = {
        "value": secret.value, "ts": now
    }
    return secret.value
```

© Copyright Microsoft Corporation. All rights reserved.

152

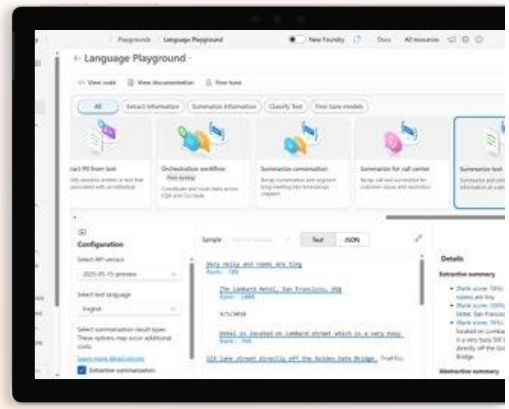
Exercise

Manage secrets with Azure Key Vault

In this exercise, you deploy an Azure Key Vault pre-loaded with sample secrets and build a Python Flask web application that demonstrates core secret management patterns using the Azure SDK.

Estimated time: 30 minutes

Start the exercise at:
https://go.microsoft.com/fwlink/?linkid=2352903



© Copyright Microsoft Corporation. All rights reserved.

153

Module 2: Manage application settings with Azure App Configuration

© Copyright Microsoft Corporation. All rights reserved.

154

Connect to App Configuration from application code

Data model and connection:

- Settings stored as key-value pairs with optional labels and content types
- Provider `load()` returns dictionary-like object for standard access

Filtering and refresh:

- `SettingSelector` filters keys by prefix pattern and label
- Sentinel key pattern triggers configuration reload without restart

```
# Filter settings by key prefix and label
selects = [
    SettingSelector(
        key_filter="Pipeline:*",
        label_filter="Production"
    )
]

config = load(
    endpoint=endpoint,
    credential=credential,
    selects=selects
)
batch_size = config["Pipeline:BatchSize"]
```

© Copyright Microsoft Corporation. All rights reserved.

155

Organize settings with labels and feature flags

Labels and composition:

- Labels create environment-specific variants of the same key
- Defaults load first, environment labels override matching keys

Feature flags:

- `FeatureManager.is_enabled()` evaluates flags at runtime
- Operators toggle features without redeployment

```
# Evaluate feature flags at runtime
config = load(
    endpoint=endpoint,
    credential=credential,
    feature_flags_enabled=True
)

feature_manager = FeatureManager(config)
if feature_manager.is_enabled("UseNewModel"):
    process_with_new_model(document)
else:
    process_with_current_model(document)
```

© Copyright Microsoft Corporation. All rights reserved.

156

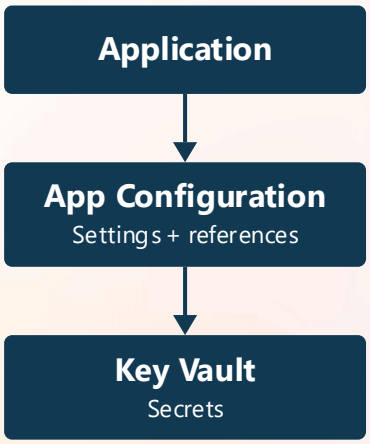
Reference Key Vault secrets from App Configuration

How references work:

- App Configuration stores a URI pointer, not the secret value
- Provider resolves secrets automatically using the reference content type
- Application accesses secrets and settings through the same dictionary

Authentication and refresh:

- Identity needs Data Reader and Key Vault Secrets User roles
- `secret_refresh_interval` controls independent secret refresh cycle



© Copyright Microsoft Corporation. All rights reserved.

157

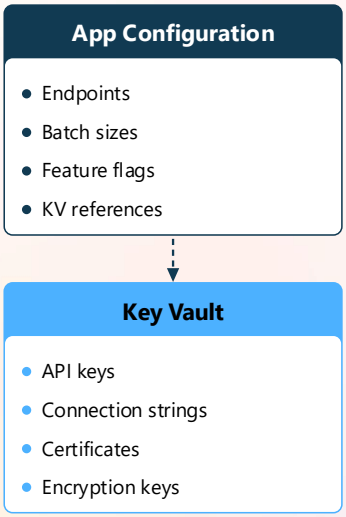
Decide what to store in App Configuration vs Key Vault

Classification criteria:

- Sensitivity: does the value grant access to a resource?
- Rotation: does it need audit trails and expiration?
- Access control: does it require per-object tracking?

Complementary architecture:

- App Configuration as single entry point for all settings
- Key Vault references for sensitive values behind the scenes



© Copyright Microsoft Corporation. All rights reserved.

158

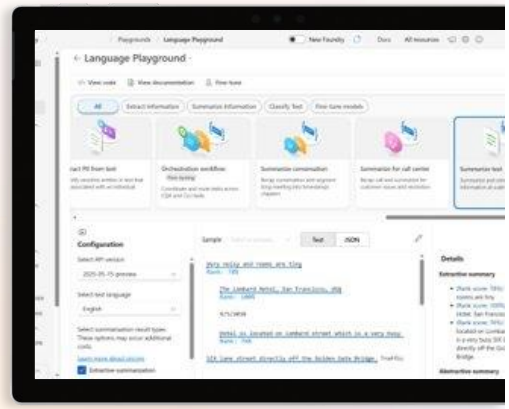
Exercise

Retrieve settings and secrets from Azure App Configuration

In this exercise, you deploy an Azure App Configuration store and Key Vault pre-loaded with sample settings and build a Python Flask web application that demonstrates core configuration management patterns using the Azure SDK.

Estimated time: 30 minutes

Start the exercise at:
https://go.microsoft.com/fwlink/?linkid=2352904



© Copyright Microsoft Corporation. All rights reserved.

159

Microsoft

Observe and troubleshoot apps on Azure

© Copyright Microsoft Corporation. All rights reserved.

160

Module 1: Instrument an app with OpenTelemetry

© Copyright Microsoft Corporation. All rights reserved.

161

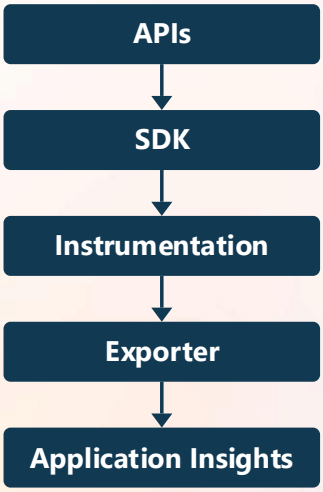
Explore OpenTelemetry and its role in observability

Observability pillars:

- Distributed tracing captures the full request path through services
- Metrics provide aggregate numerical measurements over time
- Logs record detailed, timestamped events within a service

OpenTelemetry components:

- APIs define interfaces; SDKs implement processing and export
- Instrumentation libraries auto-capture telemetry from common frameworks
- Exporters serialize and send telemetry to any compatible backend



© Copyright Microsoft Corporation. All rights reserved.

162

Add the OpenTelemetry SDK to an application

Instrumentation approaches:

- Autoinstrumentation: config-based, no code changes, limited customization
- SDK-based (manual): full control over spans, attributes, and sampling
- Distro packages SDK, exporter, and auto-instrumentation in one install

Configuration essentials:

- Connection string routes telemetry to Application Insights
- Environment variable is the recommended approach for production
- Cloud role name identifies each service on the Application Map

```python
# Configure service identity for the Application Map
from azure.monitor.opentelemetry import
configure_azure_monitor
from opentelemetry.sdk.resources import Resource

configure_azure_monitor(
    resource=Resource.create({
        "service.name": "embedding-service",
        "service.namespace": "rag-pipeline",
        "service.instance.id": "embedding-instance-1",
    })
)
```

© Copyright Microsoft Corporation. All rights reserved.

163

Configure spans and traces

Custom spans and attributes:

- Each service creates one tracer to identify its instrumentation source
- Context manager pattern handles span lifecycle and parent assignment
- Attributes add business context like model name, token count, result count

Span kinds and error handling:

- SpanKind controls table mapping: SERVER to requests, CLIENT to dependencies
- Nested spans create parent-child hierarchy without explicit references
- SDK records exceptions and sets error status on unhandled failures

```python
from opentelemetry import trace

tracer = trace.get_tracer("embedding-service")

with tracer.start_as_current_span(
    "GenerateEmbedding"
) as span:
    span.set_attribute(
        "embedding.model", "text-embedding-ada-002"
    )
    span.set_attribute(
        "embedding.token_count", token_count
    )
    embedding = generate_embedding(input_text)
```

© Copyright Microsoft Corporation. All rights reserved.

164

Export telemetry to Azure Monitor

Export pipeline:

- Direct export sends telemetry straight to Application Insights endpoint
- Signal types map to tables: requests, dependencies, traces, exceptions
- Offline storage caches data during connectivity loss for up to 48 hours

Sampling strategies:

- Fixed-percentage: collect a configurable fraction of all traces
- Rate-limited: cap traces per second (default strategy for Python Distro)
- Metrics are never sampled; only trace data is affected by sampling

© Copyright Microsoft Corporation. All rights reserved.

165

Debug distributed flows with trace data

Application Insights tools:

- Application Map shows service topology, response times, and failure rates
- End-to-end transaction view displays waterfall chart of all spans in a trace
- Custom attributes and exception details appear in span detail panels

KQL queries and AI diagnostic patterns:

- Join requests and dependencies on `operation_Id` to trace latency sources
- Query `customDimensions` for AI metrics like token counts and model names
- Watch for: embedding timeouts, search cold starts, LLM rate limiting

```
// Find slow requests and their
dependency calls
requests
| where duration > 3000
| join kind=inner (dependencies)
    on operation_Id
| project requestName = name,
    dependencyName = name1,
    dependencyDuration = duration1
| order by requestDuration desc
```

© Copyright Microsoft Corporation. All rights reserved.

166

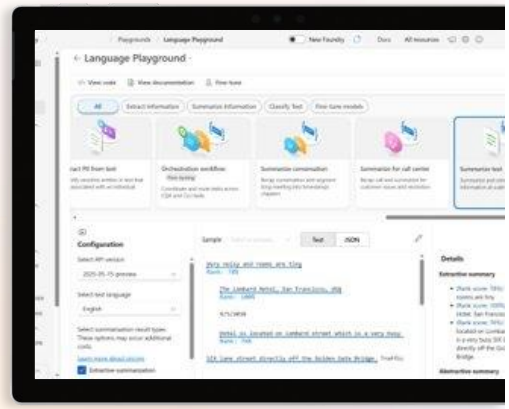
Exercise

Instrument an app with OpenTelemetry

In this exercise, you deploy an Application Insights resource and build a Python Flask web application that demonstrates OpenTelemetry instrumentation for a document processing pipeline.

Estimated time: 30 minutes

Start the exercise at:
https://go.microsoft.com/fwlink/?linkid=2353616



© Copyright Microsoft Corporation. All rights reserved.

167

Module 2: Analyze app telemetry with logs and metrics

© Copyright Microsoft Corporation. All rights reserved.

168

Write basic KQL queries (1 of 2)

Application Insights tables:

- `requests` stores incoming HTTP requests with duration, response code, success status
- `dependencies` captures outbound calls to databases, APIs, model endpoints
- `exceptions` records errors with stack traces. `traces` stores application log messages.

Core operators:

- `where` filters rows. `project` selects and renames columns.
- `take` returns arbitrary N rows. `top` returns N rows sorted by a column.
- `has` uses term indexing for fast lookups. `contains` does slower substring matching.

© Copyright Microsoft Corporation. All rights reserved.

169

Write basic KQL queries (2 of 2)

Aggregate and summarize:

- summarize groups rows with `count()`, `avg()`, `percentile()`
- `bin()` groups timestamps into fixed intervals for trend analysis

Visualize with render:

- `render timechart`, `barchart`, `piechart` for different chart types
- Including `render` preserves the visualization when pinned

```
// Aggregate requests by hour and service
requests
| where timestamp > ago(24h)
| summarize requestCount = count(),
    avgDuration = avg(duration)
    by bin(timestamp, 1h), cloud_RoleName
| render timechart
```

© Copyright Microsoft Corporation. All rights reserved.

170

Explore logs for errors and performance (1 of 2)

Investigate errors:

- Use `sum(itemCount)` instead of `count()` for accurate totals with sampling
- Group by exception type and operation to pinpoint problem areas

Correlate across services:

- Join tables on `operation_Id` to trace the full request chain
- Rename columns in subqueries to avoid ambiguous join output

```
// Sampling-aware exception counting
exceptions
| where timestamp > ago(24h)
| summarize exceptionCount = sum(itemCount)
    by bin(timestamp, 1h), type
| render timechart
```

© Copyright Microsoft Corporation. All rights reserved.

171

Explore logs for errors and performance (2 of 2)

Analyze dependency latency and failures:

- The `dependencies` table captures all outgoing calls from instrumented services
- Percentile calculations reveal latency outliers hidden by averages
- Failed dependencies grouped by target, result code, and service identify the bottleneck

Interpret result codes:

- HTTP 429: rate limiting. Adjust throughput or implement retry policies.
- HTTP 500: server-side failure in the dependency itself
- Timeout with no result code: network issues or overloaded service

© Copyright Microsoft Corporation. All rights reserved.

172

Build dashboards for app telemetry

Azure dashboards for monitoring:

- Customizable portal surfaces combining tiles from multiple Azure resources
- Optimized for at-a-glance visibility with auto-refreshing tiles
- Can span multiple subscriptions and resource groups on a single dashboard

Pin metrics charts:

- Navigate to Application Insights Metrics pane, select a metric, apply filters and splitting
- Common metrics: server response time, failed requests, dependency failures, request rate
- Aggregation type matters: average for typical behavior, sum for total events, max for worst-case

© Copyright Microsoft Corporation. All rights reserved.

173

Set alerts for app failures and anomalies (1 of 2)

Alert rule components:

- Scope and condition: Resource monitored and signal/threshold logic
- Action group and severity: Notifications, actions, urgency level (0-4)

Alert types:

- Metric alerts: Evaluate resource metrics at regular intervals
- Log search alerts: Run KQL queries for complex detection logic

```
// Alert query: detect high failure rate
requests
| where success == false
| summarize failedCount = count()
    by cloud_RoleName
| where failedCount > 10
```

© Copyright Microsoft Corporation. All rights reserved.

174

Set alerts for app failures and anomalies (2 of 2)

Action groups for notifications:

- Reusable across multiple alert rules (up to five groups per rule)
- Email, SMS, Azure mobile app push, voice call notifications
- Automated actions: Azure Function, Logic App, webhook, Automation Runbook, Event Hub

Smart detection for anomaly identification:

- Machine learning identifies unusual patterns without manual threshold configuration
- Failure anomaly detection compares current failure rate to historical baseline
- Performance anomaly detection catches gradual degradation. Best used alongside manual alert rules.

© Copyright Microsoft Corporation. All rights reserved.

175

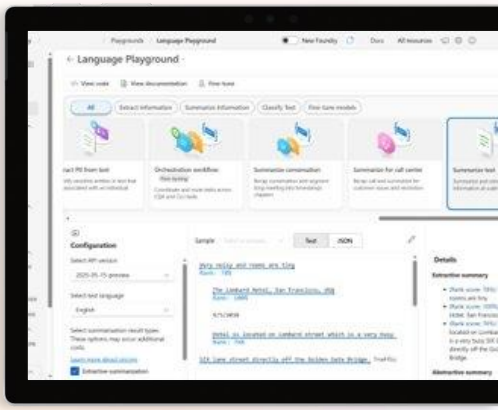
Exercise

Query logs with KQL

In this exercise, you deploy an Application Insights resource, run a Python script that generates sample request, dependency, and exception telemetry using OpenTelemetry, then write KQL queries in the Azure portal Logs blade to investigate application health.

Estimated time: 30 minutes

Start the exercise at:
https://go.microsoft.com/fwlink/?linkid=2353415



© Copyright Microsoft Corporation. All rights reserved.

176

Thank you for attending this course

Here are some reminders



Celebrate your new skills

- Redeem your achievement
- Share with us and your network



Let us know how we did

- Give us your feedback
- Survey will be sent via email



Accelerate your success with Microsoft Credentials

- Explore additional resources to help prepare
- Schedule your credential

Tag us social: #AlwaysLearning

177

What is next on your learning journey?

AI-300: Operationalize Machine Learning and Generative AI Solutions

Description

As a candidate for this Microsoft Certification, you should have subject matter expertise in setting up infrastructure for machine learning operations (MLOps) and generative AI operations (GenAIOps) solutions on Azure, together referred to as AI operations (AIOps).

You need experience training, optimizing, deploying, and maintaining traditional machine learning models by using Azure Machine Learning, in addition to experience deploying, evaluating, monitoring, and optimizing generative AI applications and agents by using Microsoft Foundry

Topics covered

EXAMPLE

- Design and implement an MLOps infrastructure
- Implement machine learning model lifecycle and operations
- Design and implement a GenAIOps infrastructure
- Implement generative AI quality assurance and observability
- Optimize generative AI systems and model performance

Find more information about this course at: learn.microsoft.com/training/courses/browse

178

Thank You

© Copyright Microsoft Corporation. All rights reserved.

179